The tools and techniques of R programming
Lesson-9
Graphical Representation of Data

By
Prof.Dr. A.B.Chowdhury,HOD,CA

Techno India University, West Bengal,India
Email-Id:hod.ca@technoindiaeducation.com

October 31, 2023

Objectives of this Lesson

Graphical Representation is a method to show and represent values pictorially with increments, decrements and comparisons to either make predictions or to show a report depicting how certain situation changes from the past to the present.It is the visual display of data through charts and graphs. These graphical representations are used by business analyst, traders and investors, mathematics professors, scientists, business and financial advisors, marketers, informers, architects, army or navy officers for comparative studies with a glance without making time-consuming computations with the available data.

Data Visualization is a powerful way to simplify the complexity in a vast amount of data and present it in a form which is easily comprehensible even by a novice.A pictorial presentation turns data into actionable information, and information into the knowledge insight. From early caveman paintings to today's sophisticated information dashboards, human beings have utilized data viz in some form or the other since time immemorial.In the words of Edd Dumbill,"**The art and practice of visualizing data is becoming ever more important in bridging the human-computer gap to mediate analytical insight in a meaningful way**". We may finally conclude with the words of John Tukey: "**The greatest value of a picture is when it forces us to notice what we never expected to see**".

Forms of graphical presentations

R provides the following forms of pictorial presentation of data:

1. Pie Charts/graphs
2. Bar Charts/graphs
3. Stacked Bar Charts/graphs
4. Histograms
5. Scatter Plots
6. Line Charts
7. Box Plots

We shall now study the different forms one by one.

Pie Charts

A pie chart is also known as a **pie graph**.It is a specialized graph used in statistics for a comparative graphical display of proportionate values of a numeric variable (a dependent variable) against a categorical variable (an independent variable). **The independent variable is plotted around a circle in either in a clockwise direction or in an anticlockwise direction.**

Pie Charts–Contd.

The percentage values of the dependent variable is shown through an arc depicted by constructing radial lines from its ends to the centre of the arc creating a wedge-shaped **"slice"**.

The independent variable can have a finite number of values; whereas, the dependent variable can have any value from 0 to 100 percent to represent a portion out of 360 degrees in the circle. The lesser the number of values in the independent variable, the better readable and understandable is the pie chart. **This type of graphical presentation is ideal and effective for giving the reader a quick idea of the proportionate distribution of the data against the categorical variable under study and hence used extensively in the business world and mass media.**

However, this type of graphical representation is criticized for some of its drawbacks. Firstly,as the number values of the categorical variable increases, understanding of the variations in the distribution of data by comparing the slices of the dependent variable decreases.Secondly, relative comparisons among a number of pie charts under study are not well understandable.

Pie Charts–Contd.

3D pie chart

For aesthetic reasons, people sometimes like to draw a three dimensional pie chart, although the third dimension does not improve the readability of the data rather it gives a distorted effect of the perspective associated with the third dimension.

Exploded pie chart

A pie chart with one or more slices separated from the rest of the disk is called an exploded pie chart. It is used either to highlight some specific sector(s) to draw special attention or to display the smaller sections having a small proportion of data.

R provides the **pie() function** for drawing a pie chart readily. The syntax of the function is given below:

pie(x,labels,edges,radius,clockwise, init.angle,density,col,border,lty,main. . .)

where, x is a vector containing nonnegative numeric values that will be plotted. labels is used to generate labels for the slices. Its default value is names(x) and written as labels=names(x) edges is used to specify a numeric value to indicate the number of segments used to draw the outside of the pie.

Pie Chart–Contd.

Its default value is 200 and written as edges=200.
radius is used to mention a numeric value indicating how bog the pie should be. Its default value is 0.8 written as radius=0.8. If the specified value is greater than 1, then parts of the pie are cut off.
clockwise option holds a logical value TRUE or FALSE to indicate whether the slices are to be drawn clockwise or counterclockwise. Its default value is FALSE.
init.angle is a numeric value (in degrees) to mention the starting angle for the slices . Its default value value is 90,in case of clockwise =TRUE, otherwise,0. density it indicates the density of shading lines per inch. Density =NULL points out that no such shading lines are required.
angle is used to mention the slope of the shading lines. It is numeric value in degrees..The default value is 45
col is a numeric vector mentioning the colours to be used for the slices.If col=NULL, then a set of six pastel colours is used.Its default value is NULL.
border implies the arguments passed to the polygon function to draw each slice.Default value is NULL.
lty points out the line type used to draw each slice.Its default value is NULL.
main is used to specify a character string to represent the title of the pie chart. Its default value is NULL
... Additional arguments, if required.

Illustrative Examples of PIE charts

Let us illustrate a pie chart . Let us suppose that in the End Semester Examination(ESA) of a university the grade obtained by the students taking the R examinations is as shown below:

GRADES:	A	B	C	D	F
Number of students :	50	30	10	7	3

We would like to show the result through a pie chart.

We define:

X=c(50,30,10,7,3)

Grades=c('A','B','C','D','F')

names(X)=Grades

pie(X,labels=names(X),radius=0.9,clockwise=FALSE)

This command will produce the following pie graph:

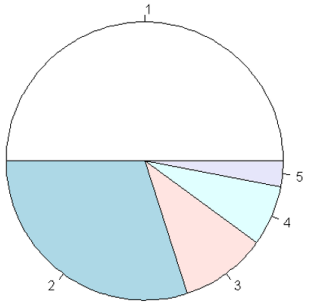


Figure: Pie Chart with the Grades obtained

Illustrative Pie Charts-Contd.

If we want to add labels to each of the slices, we can declare a vector of labels as follows:

Grades=c("A","B","C","D","F")

Now, we can use it as follows:

pie(X,labels=grades,radius=0.9,clockwise=FALSE)

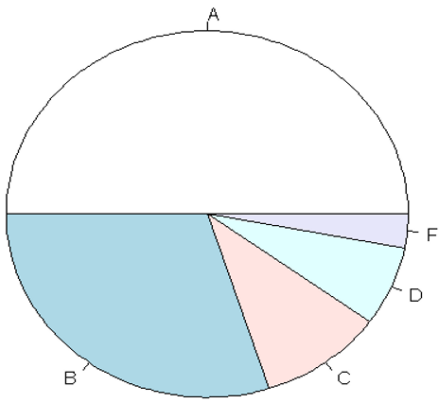


Figure: Pie Chart with slices labelled with Grades

Let us now define a vector of colors as shown below:

cols=c("green","violet","blue","black","red")

Now, we can use this vector in the pie function to get the desired effect of the defined colors as follows:

Illustrative Pie Charts-Contd.

pie(X,labels=grades,radius=1,clockwise=FALSE,col=cols)

The output of the above statement will be figure shown below:

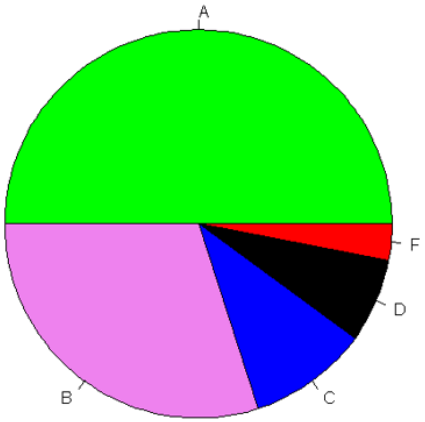


Figure: Pie Chart with slices labelled with Grades and filled with desired colours

We can use the vector directly in the function also without defining it separately. Next thing that we would like to have in our graph is to add legends to the graph explaining the meaning of the colours. This can be done by defining a **legend()** function as shown below:

legend(position,labels,fill)

e.g.;

legend("topright",fill=cols,grades)

Illustrative Pie Charts-Contd.

This will generate the following figure:

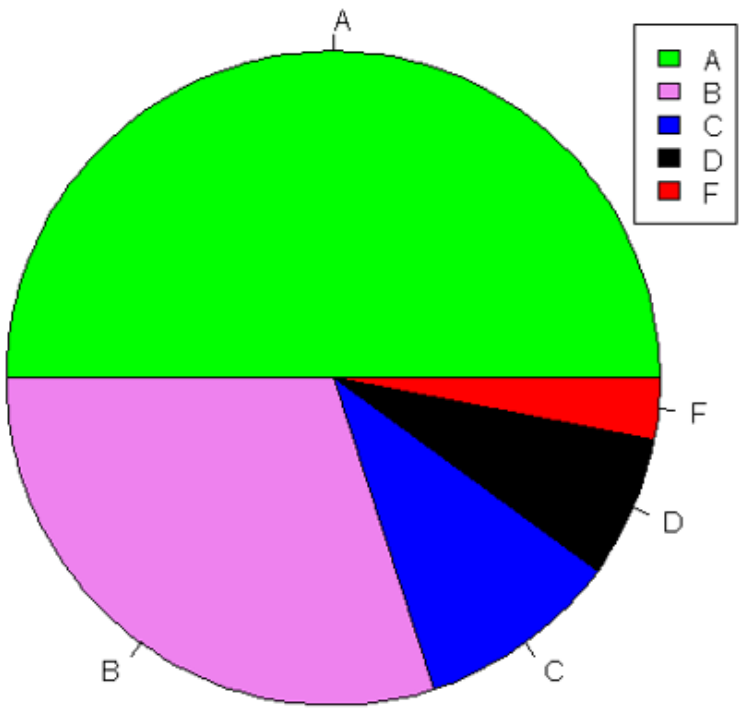


Figure: Pie Chart with slices labelled with Grades and filled with desired colours along with legends

The legends can also be changed according to our choice as shown below:

Illustrative Pie Charts-Contd.

```
y=c("Best","Good","Mediocre","Bad","very Bad")
pie(X,labels=grades,radius=0.7,clockwise=FALSE,col=cols)
legend("bottomleft",fill=cols,y)
```

The generated figure will be as shown below:

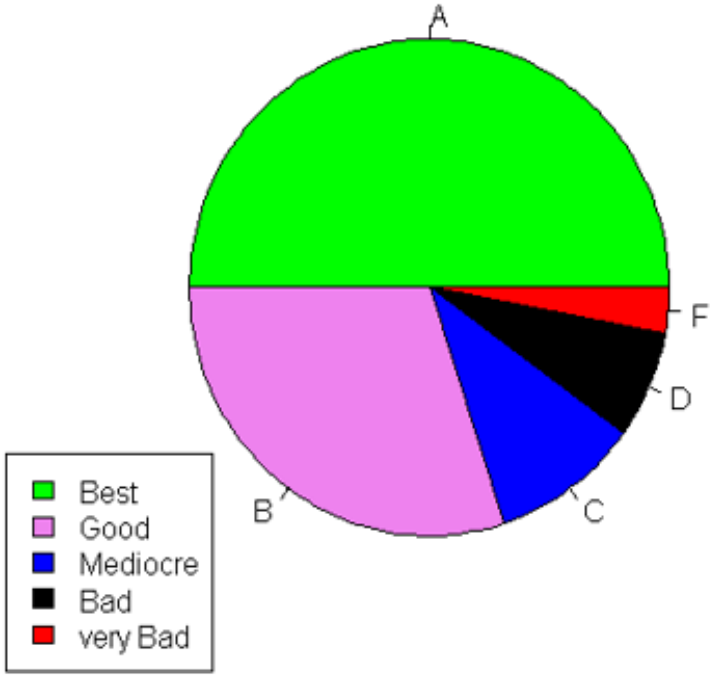


Figure: Pie Chart with slices labelled with Grades and filled with desired colours along with changed legends

Illustrative Pie Charts-Contd.

We can also add headings to the graph by writing any string under the main option. This is illustrated below:

pie(X,labels=grades,radius=0.7,clockwise=FALSE,col=cols,main="Presentation of MCA2,TIU,WB Results of ESA on R")

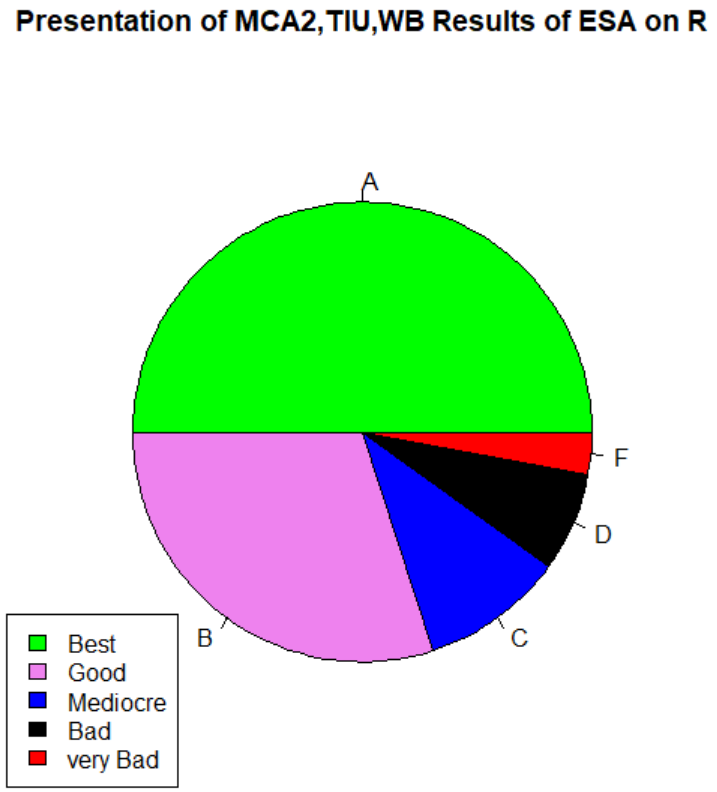


Figure: Pie Chart with slices labelled with Grades and filled with desired colours along with legends and graph headings

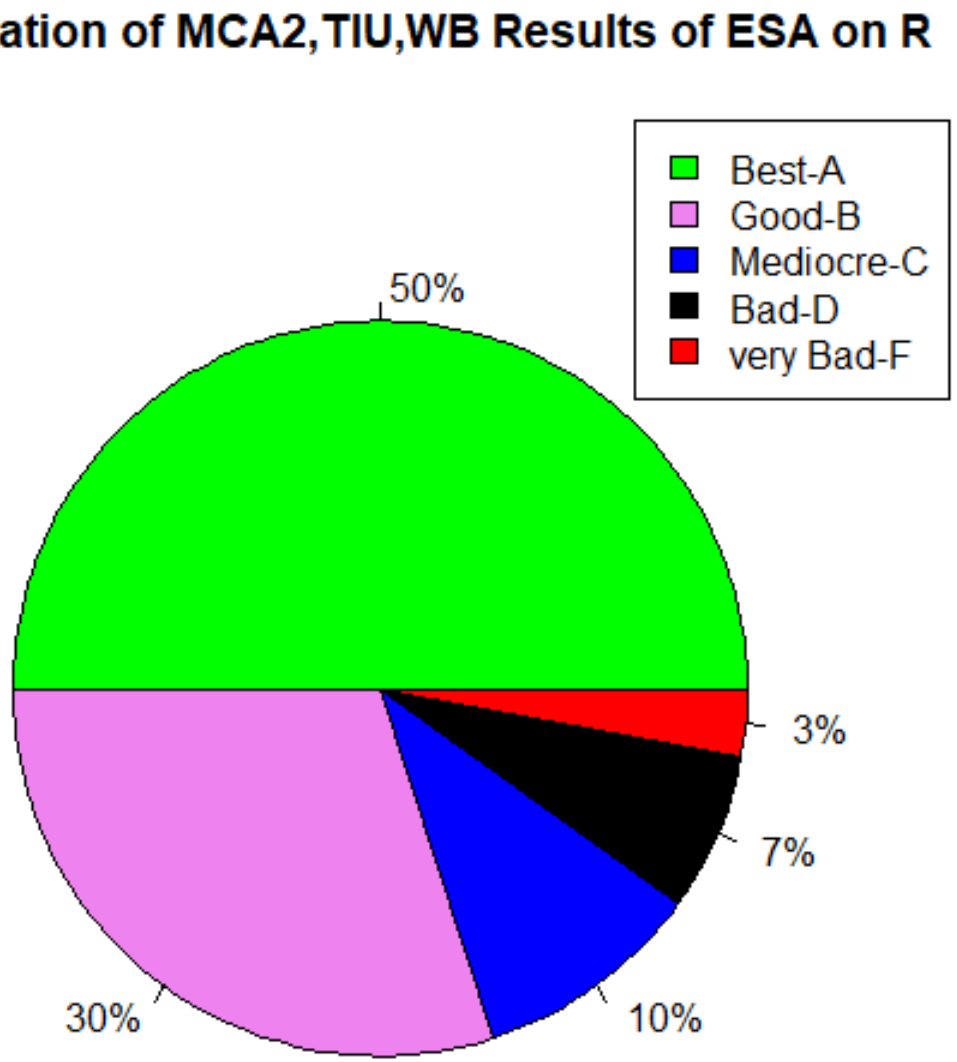
Improvised Presentation of the preceding Pie Chart

We can further improvise the pie graph shown above by showing the percentages for each grade inserted over the slices with the statements shown below:

```
>percentage=round(X/sum(X)*100)
>lbells=""
>lbells = paste(lbells, percentage) #This is to add percents to labels
>lbells =paste(lbells,"%",sep="") # This is to add the' %' sign to labels
>y=c("Best-A","Good-B","Mediocre-C","Bad-D","very Bad-F")
>pie(X,labels= lbells,radius=0.7,clockwise=FALSE,col=cols,
main="Presentation of MCA2,TIU,WB Results of ESA on R")
>legend("topright",fill=cols,y)
```

The generated chart is shown below:

The Improvised Pie Chart



Illustrative Pie Charts-3D Pie Chart

If we want to draw a pie chart with 3 dimensions, we need to use one additional package called **plotrix**. The package **plotrix** has a function called **pie3D()** that is used for this purpose. Such a pie chart is also called an **Exploded Pie Chart**.No, assuming that the package **plotrix** has been installed properly, we follow the following steps:

>**library (plotrix)** # This is to load the required library. Now,with the defined values as shown above,we use the following statement:

>**pie3D(X,labels=Grades,explode=0.1,col=cols,main="Presentation of MCA2,TIU,WB Results of ESA on R")**

It may be noted that the two options used earlier, namely, 'radius' and 'clockwise', are not available here for obvious reasons.

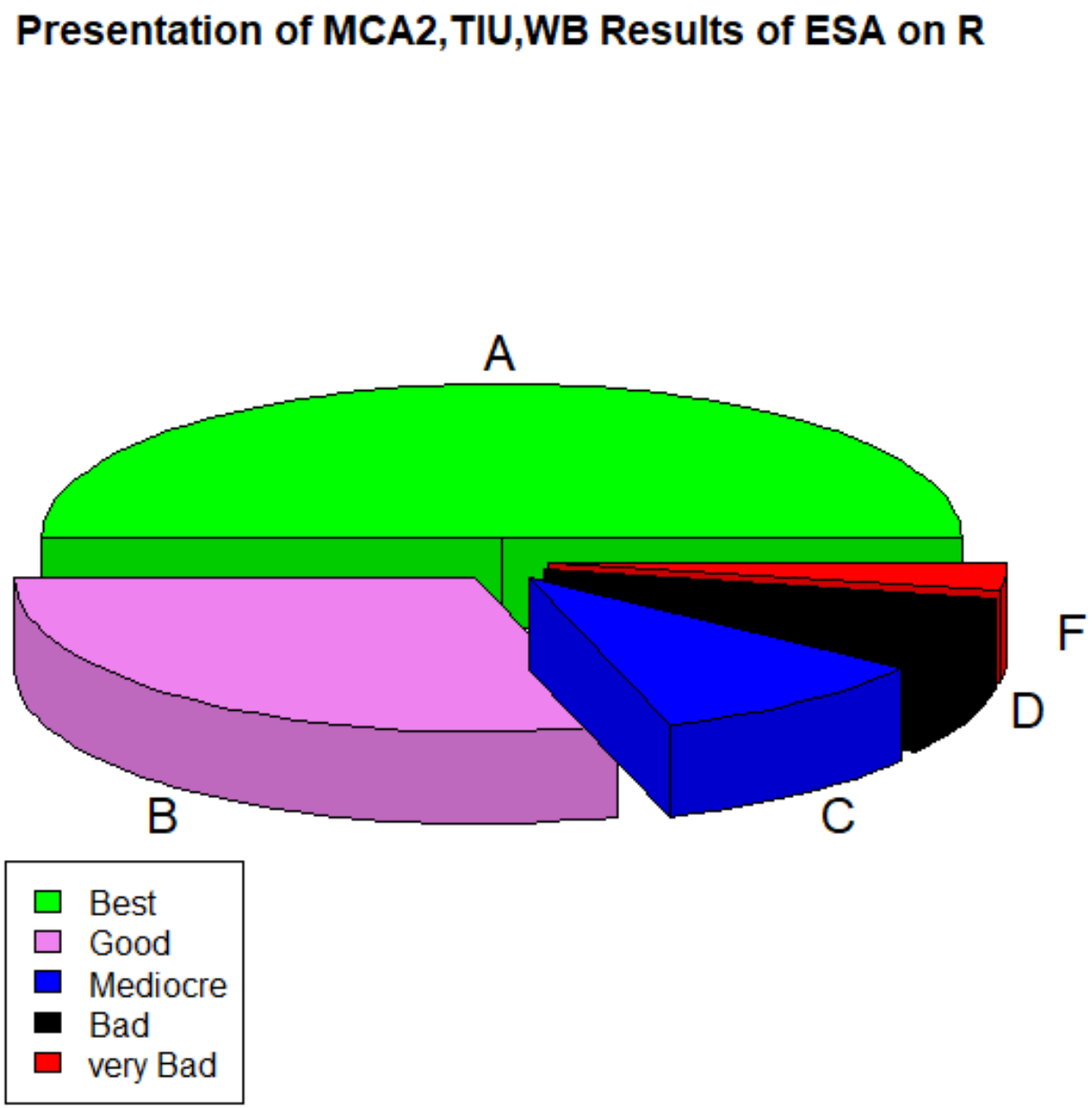
We can assign a file name to this chart with a statement like one of the following,depending on our requirement of file type:

png(file = "exploded_pie.jpg") or **png(file = "exploded_pie.png")**

After the plotting of the chart with the **pie3D() statement**, we save the file by issuing the following command:**dev.off()**

Now, we show the generated chart as under:

Exploded Pie Chart for illustration



Concepts of BAR GRAPH/CHART

Bar Graph: The word **'bar'** in a bar graph or a bar chart as it is called sometimes, originates from the meaning of a bar that implies a long rigid piece of wood, metal, or similar material, typically used as an obstruction, fastening, or weapon; e.g. an aluminium bar.

A **Bar Graph (or a bar chart)** is a pictorial presentation of categorical or qualitative statistical data shown either horizontally or vertically in order to allow viewers to compare the data values displayed.The length of the bar is proportional to the value of the variable The displayed data relate to things like amounts, characteristics, times and frequency etc.The data correspond to two types of variables, namely, an independent variable and a dependent variable. The values of the independent variable are usually plotted along the x-axis(horizontal axis) and that of the dependent variable are usually plotted along the y-axis(vertical axis). The independent variable can attain only certain discrete values, whereas, the values of the dependent variable may be either discrete or continuous.

Vertical bar graphs are drawn most commonly which are also known as column graphs.In a **column graph**, values of the independent variable are plotted along a horizontal axis from left to right. Function values are shown as shaded or colored vertical bars of equal thickness extending upward from the horizontal axis to various heights.

Concepts of BAR GRAPH/CHART–Contd.

In a horizontal bar graph, the independent variable is plotted along a vertical axis from the bottom up. The values are shown as shaded or colored horizontal bars of equal thickness extending toward the right, with their left ends vertically aligned.

From the graphical representation of information in a bar graph, we can make generalizations and conclusions quickly and easily. A typical bar graph should have labels and scale apart from axes and bars. The bars on a bar graph may be of the same colors, but different colors may be used to distinguish between groups to make the data better readable and understandable. **For the interpretation of a bar chart, we first look at the tallest bar and then at the shortest bar. The titles, if any, should be consistent and self-explanatory.**

Bar graphs can be classified as either single, or grouped or stacked. Single bar graphs are used to convey the discrete value of the item for each category shown on the opposite axis. An example may be a representation of the number of Indian males participated in the last five Olympic games.

A grouped or clustered bar graph is used to represent discrete values for more than one item that share the same category. An example may be, using the single bar example above and introduce the number of Indian females in the last five Olympic games.

Concepts of BAR GRAPH/CHART–Contd.

Finally, if each of the bars is divided into sub-parts that represent the discrete value for items that represent a portion of a whole group, then it is called a **stacked bar graph**.An example may be to represent the actual data for males in the example above and then scale each discrete value as a part of the whole for each bar. Again color coding should be done to make the graph readable.

Bar graphs serve the purposes as stated below:

- **Depict** each category of data in a frequency distribution.
- **Show** relative numbers or proportions of multiple categories.
- **Summarize** a large set of data in visually pleasing and understandable form.
- **Clarify** trends better than values in tables
- **Estimate** the key values at a glance.
- **Permit** a visual examination of the correctness and reasonableness of calculations.

Drawing Bar Charts

R provides the function **barplot()**. The syntax of the function is given below:

barplot(height,width=1,space=NULL, names.arg=NULL, legend.text=NULL,beside=FALSE, horiz=FALSE, density=NULL, angle=45,col=NULL,border=par("fg"), main=NULL,sub=NULL,xlab=NULL, ylab=NULL,xlim=NULL, ylim=NULL, xpd=TRUE,log="", axes=TRUE, axisnames=TRUE, cex.axis=par("cex.axis"), cex.names=par("cex.axis"),inside=TRUE, plot=TRUE,axis.lty=0,offset=0,add=FALSE, arg.legend=NULL,...)

Where,

height is either a vector or matrix of values describing the bars which make up the plot. For instance,If weight is a vector, the plot consists of a sequence of rectangular bars with weights given by the values in the vector. If weight is a matrix and beside is FALSE then each bar of the plot corresponds to a column of weight, with the values in the column giving the weights of stacked sub-bars making up the bar. If weight is a matrix and beside is TRUE, then the values in each column are juxtaposed (placed or dealt with close together for contrasting effect rather than stacked)

width is optional vector of bar widths. Re-cycled to length the number of bars drawn. Specifying a single value will have no visible effect unless **xlim** is specified.

space is the amount of space (as a fraction of the average bar width) left before each bar. May be given as a single number or one number per bar. If height is a matrix and beside is TRUE, space may be specified by two numbers, where the first is the space between bars in the same group, and the second the space between the groups. If not given explicitly, it defaults to c(0,1) if height is a matrix and beside is TRUE, and to 0.2 otherwise.

names.arg is a vector of names to be plotted below each bar or group of bars. If this argument is omitted, then the names are taken from the names attribute of weight if this is a vector, or the column names if it is a matrix.

Explanations of the arguments in barplot() contd.

legend.text is a vector of text used to construct a legend for the plot, or a logical indicating whether a legend should be included. This is only useful when weight is a matrix. In that case given legend labels should correspond to the rows of weight; if legend.text is true, the row names of weight will be used as labels if they are non-null.
beside is a logical value. If FALSE, the columns of height are portrayed as stacked bars, and if TRUE the columns are portrayed as juxtaposed bars.
horiz is a logical value. If FALSE, the bars are drawn vertically with the first bar to the left. If TRUE, the bars are drawn horizontally with the first at the bottom.
density is a vector giving the density of shading lines, in lines per inch, for the bars or bar components. The default value of NULL means that no shading lines are drawn. Non-positive values of density also inhibit the drawing of shading lines.
angle is the slope of shading lines, given as an angle in degrees (counter-clocwise), for the bars or bar components.
col is for the colors for the bars or bar components. By default, grey is used if height is a vector, and a gamma-corrected grey palette if height is a matrix. border is the color to be used for the border of the bars. We use border = NA to omit borders. If there are shading lines, then border = TUE means the use of the same colour for the border as for the shading lines.
main,sub is used to specify the overall and sub title for the plot.
xlab is used to specify a label for the x-axis.
ylab is used to specify a label for the y axis.
xlim, ylim are used to mention the limits for the x and y axis respectively.
xpad is logical to specify whether bars should be allowed to go beyond the available region or not.
log contains a string pointing out if axis scale should be logarithmic.
axes is logical ,if TRUE, a vertical(or horizontal,if horiz is TRUE) axis is drawn.
axisnames is logical. If TRUE, and if there are names.arg , the other axis is drawn (with lty = 0) and labeled.
cex.axis is expansion factor for axis names (bar labels)
inside is meant for logical. If TRUE, the lines which divide adjacent (non-stacked!) bars will be drawn. Only applies when space = 0 (which it partly is when beside = TRUE).
plot is logical. If FALSE, nothing is plotted
axis.lty is the graphics parameter lty applied to the axis and tick marks of the categorical (default horizontal) axis. It may be noted that by default, the axis is suppressed.
offset is a vector indicating how much the bars should be shifted relative to the x axis.
add is logical specifying if bars should be added to an already existing plot; defaults to FALSE.
args.legend mentions a list of additional arguments to pass to legend(); names of the list are used as argument names. Only used if legend.text is supplied.
..... implies arguments to be passed to/from other methods. For the default method these can include further arguments (such as axes, asp and main) and graphical parameters (par) which are passed to **plot.window()**, **title()** and **axis.**

Illustration of a bar Graph

Let us now make use of some of the basic arguments to draw a bar graph. Let us consider a set of data consisting of number of students admitted in a certain university in 2021 in different noticeable streams of study. So, we can define the vectors of the data values as follows:

a=c(295,235,172,165) # Number of admitted students in four different streams

s=c("BCA","B.Tech(CSE)","B.Tech(ECE)","B.Tech(EE)")

Here s is defined to contain the names of the streams corresponding to the admission counts.

We can now draw a bar graph by specifying a suitable heading(Optional) and the colors in the bars along with the separating colours as follows:

barplot(a,names.arg=s,xlab="Streams",ylab="No. of students got admitted",col="green",main="Student Strength in 2018", border="red")

The bar graph corresponding to the above specification is shown in the figure below:

An Illustrative Bar Graph

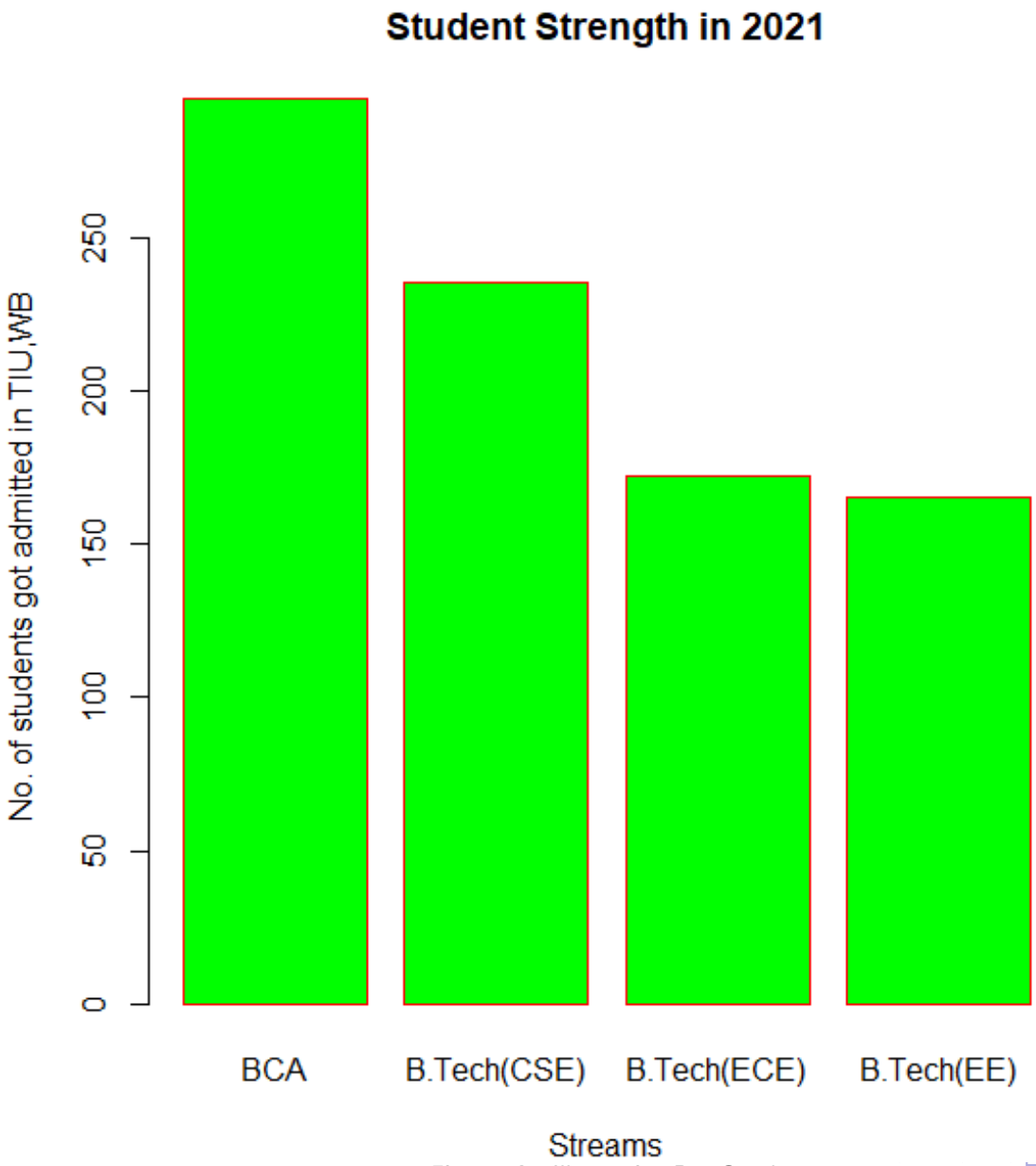


Figure: An Illustrative Bar Graph

Group Bar Chart and Stacked Bar Chart

As discussed earlier, we can create bar chart with groups of bars and stacks in each bar by using a matrix as input values. We know that more than two variables are represented as a matrix which is used to create the group bar chart and stacked bar chart. We want to create such a bar graph to depict a comparative study of admissions in the last four years in **TIU,WB**,in four significant streams of study.The data taken are,of course, some imaginary values.The steps are shown below:

```
>colors=c("blue","orange","brown","green")
>streams=c("BCA","B.Tech(CSE","B.Tech(ECE)","B.Tech(EE)")
>years=c("2015","2016","2017","2018")
>data=matrix(c(30,450,350,250,195,375,245,180,325,295,225,175,
295,275,173,160),byrow=TRUE,nrow=4)
```

Then the following command is issued:

```
>barplot(data,main = "A comparative Study of Admissions in TIU,WB"
, names.arg = streams, xlab = "Streams", ylab = "No. of Admitted
Students", col = colors)
```

Bar chart with groups of bars and stacks in each bar

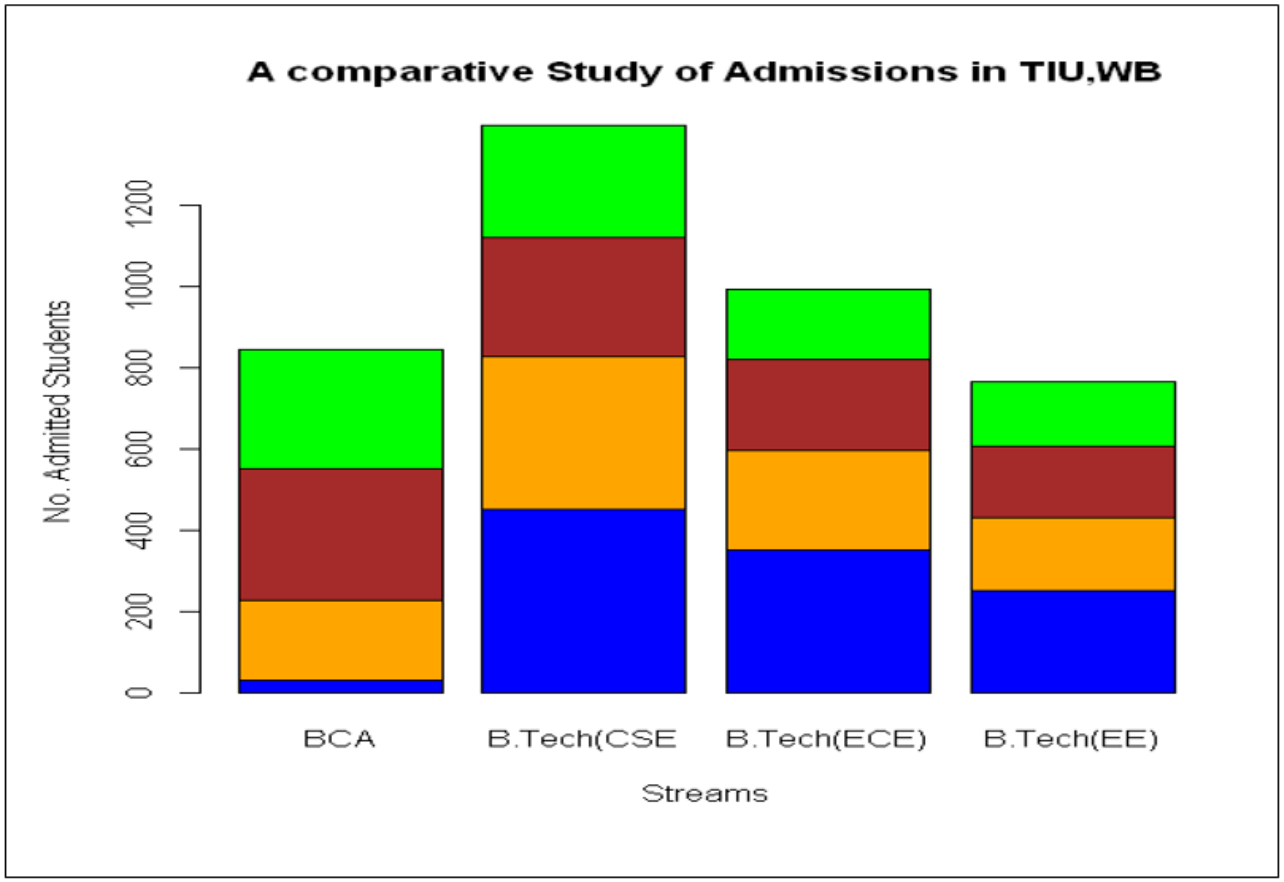


Figure: An Illustrative Stacked Bar Graph

An Illustrative Bar Chart with multiple sets of data

Let us now draw another bar graph, this time, taking data of several variables where we want to plot the magnitude of each of the values of each variable.. Let us take five variables , for instance, named as A,B,C, D and E as defined below and execute the following commands to attach the set of data defined.We next display them,define the desired colours and finally execute the statement for drawing bar chart.

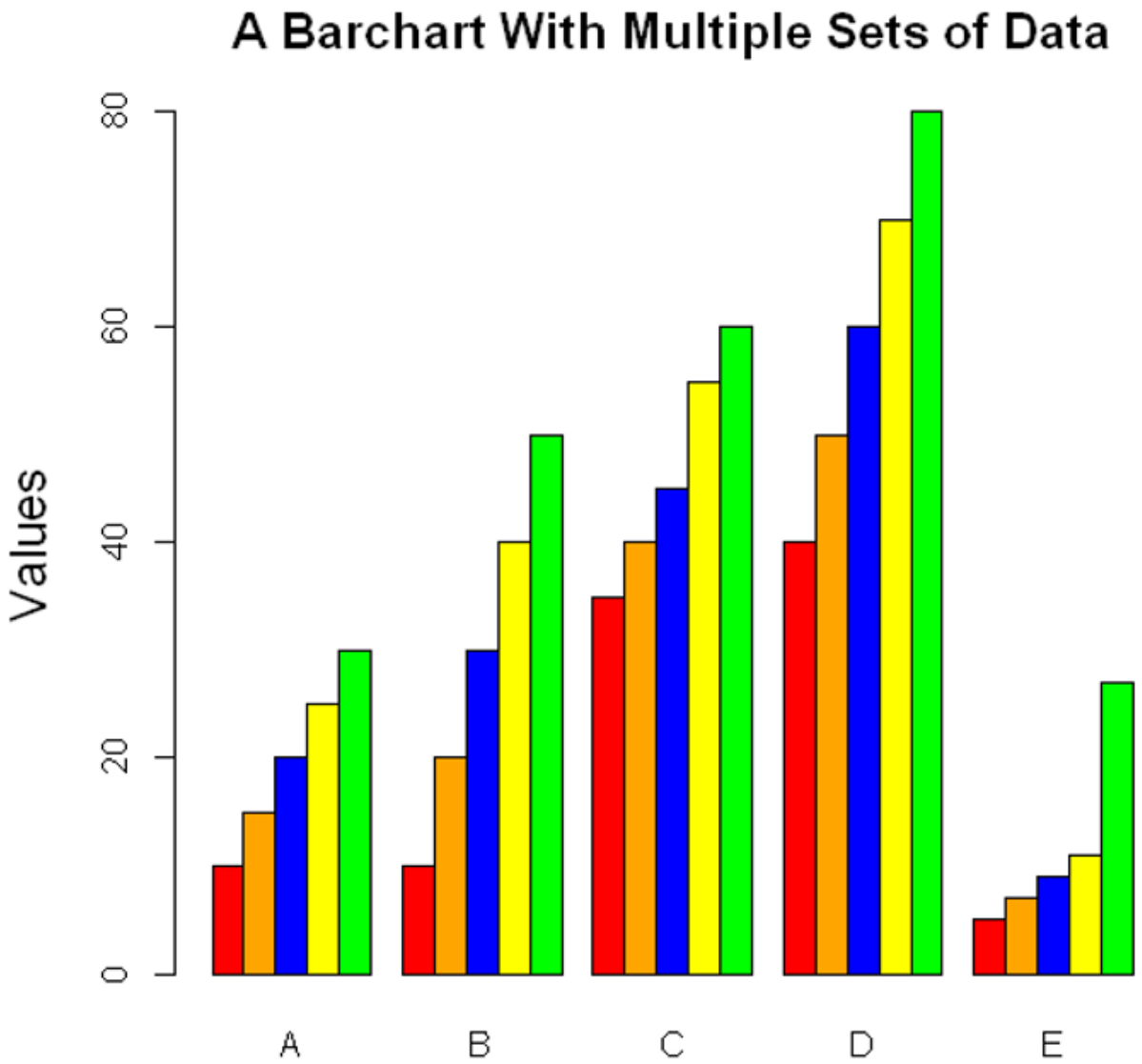
```
>data_set=structure(list(A=c(10L,15L,20L,25L,30L),B=c(10L,20L,30L,
40L,50L),C=c(35L,40L,45L,55L,60L),D=c(40L,50L,60L,70L,80L),
E=c(5L,7L,9L,11L,27L)),Names=c("A","B","C","D","E"),class=
"data.frame",row.names=c(NA,-5L))
>attach(data_set)
>print(data_set)
```

	A	B	C	D	E
1	10	10	35	40	5
2	15	20	40	50	7
3	20	30	45	60	9
4	25	40	55	70	11
5	30	50	60	80	27

```
>colours = c("red", "orange", "blue", "yellow", "green")
>barplot(as.matrix(data_set), main="A Bar chart With Multiple Sets of Data", ylab = "Values", cex.lab = 1.5,
cex.main = 1.4, beside=TRUE, col=colours)
```

The following Bar chart is generated by the above set of commands:

Figure of an Illustrative Bar Chart with multiple sets of data



Legends in the Illustrative Bar Graph with multiple sets of data

Now, we issue the following command to attach legends explaining the colours of the bars by setting bty="n" to control the border of the legend :
>**legend("topleft", c("First","Second","Third","Fourth","Fifth"), cex=1.3, bty="n", fill=colours)**
The following figure shows the graph with the legends:

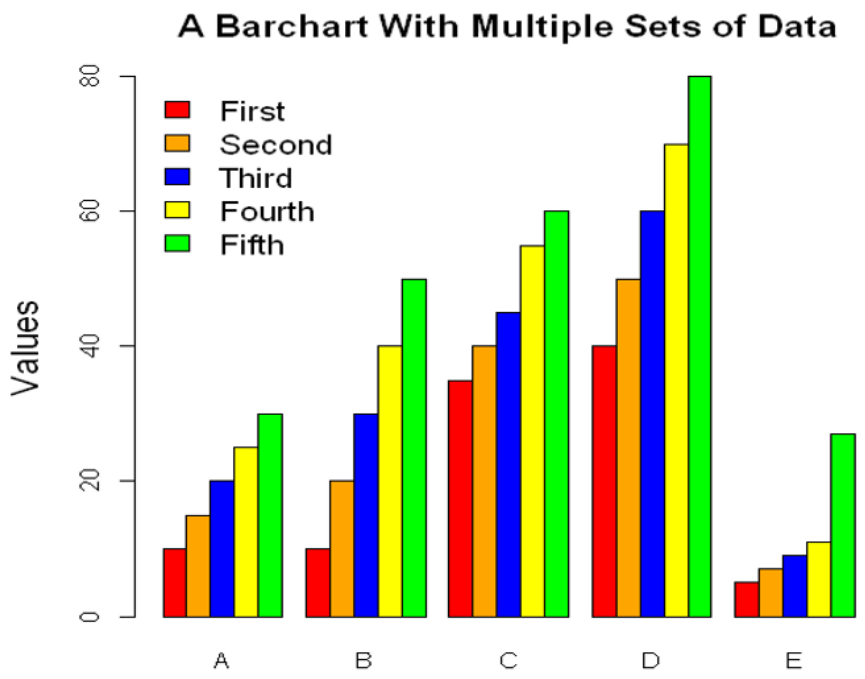


Figure: An Illustrative Bar Graph with multiple sets of data having legends

Horizontal Bar Chart for illustration

By setting the **horiz** parameter to TRUE, we get the bars in horizotal position as shown in the figure below. Here, we should also change the position of the legends so that we get it displayed at the "bottom right" location. The following set of commands have been used to get the graph drawn:

>barplot(as.matrix(data_set), main="A Barchart With Multiple Sets of Data", ylab = "Values", cex.lab = 1.5, cex.main = 1.4, beside=TRUE, col=colours,horiz=TRUE)

>legend("bottomright", c("First","Second","Third","Fourth","Fifth"), cex=1.3, bty="n", fill=colours)

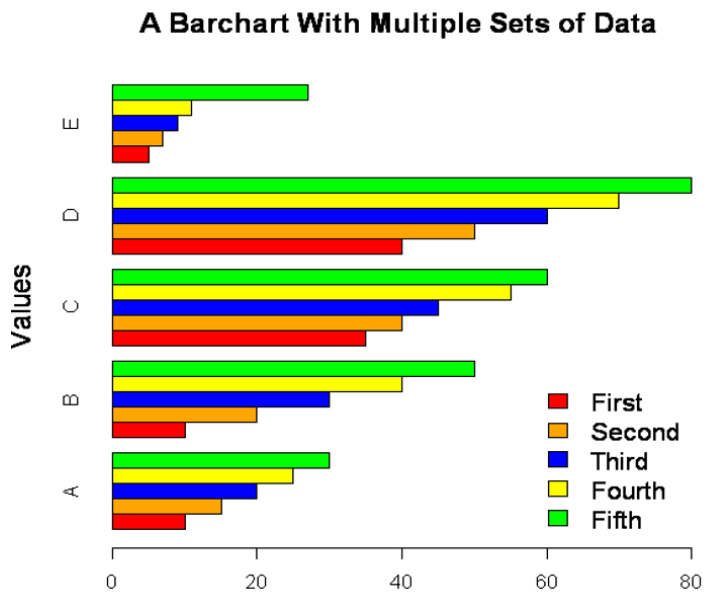


Figure: An Illustrative Bar Graph with bars in horizontal position drawn with multiple sets of data having legends

Syntax of legend()

Since the legend in a graph plays a vital role, we state the complete syntax of the **legend() function** is as shown below to enable its use in a flexible manner :

legend(x, y = NULL, legend, fill = NULL, col = par("col"), border = "black", lty, lwd, pch, angle = 45, density = NULL, bty = "o", bg = par("bg"), box.lwd = par("lwd"), box.lty = par("lty"), box.col = par("fg"), pt.bg = NA, cex = 1, pt.cex = cex, pt.lwd = lwd, xjust = 0, yjust = 1, x.intersp = 1, y.intersp = 1, adj = c(0, 0.5), text.width = NULL, text.col = par("col"), text.font = NULL, merge = do.lines has.pch, trace = FALSE, plot = TRUE, ncol = 1, horiz = FALSE, title = NULL, inset = 0, xpd, title.col = text.col, title.adj = 0.5, seg.len = 2), where

x, y the x and y co-ordinates to be used to position the legend. They can be specified by keyword or in any way which is accepted by xy.coords: See 'Details'.

legend a character or expression vector of length ≥ 1 to appear in the legend. Other objects will be coerced by as.graphicsAnnot.

fill if specified, this argument will cause boxes filled with the specified colors (or shaded in the specified colors) to appear beside the legend text.

col the color of points or lines appearing in the legend.

border the border color for the boxes (used only if fill is specified).

lty, lwd the line types and widths for lines appearing in the legend. One of these two must be specified for line drawing.

pch the plotting symbols appearing in the legend, as numeric vector or a vector of 1-character strings (see points). Unlike points, this can all be specified as a single multi-character string. Must be specified for symbol drawing.

angle angle of shading lines.

density the density of shading lines, if numeric and positive. If NULL or negative or NA color filling is assumed.

bty the type of box to be drawn around the legend. The allowed values are "o" (the default) and "n".

bg the background color for the legend box. (Note that this is only used if bty != "n".)

box.lty, box.lwd, box.col the line type, width and color for the legend box (if bty = "o").

pt.bg the background color for the points, corresponding to its argument bg.

cex character expansion factor relative to current par("cex"). Used for text, and provides the default for pt.cex.

pt.cex expansion factor(s) for the points.

pt.lwd line width for the points, defaults to the one for lines, or if that is not set, to par("lwd").

xjust how the legend is to be justified relative to the legend x location. A value of 0 means left justified, 0.5 means centered and 1 means right justified.

yjust the same as xjust for the legend y location.

x.intersp character interspacing factor for horizontal (x) spacing.

Syntax of legend()–Contd.

y.intersp the same for vertical (y) line distances.
adj numeric of length 1 or 2; the string adjustment for legend text. Useful for y-adjustment when labels are plotmath expressions.
text.width the width of the legend text in x ("user") coordinates. (Should be a single positive number even for a reversed x axis.) Defaults to the proper value computed by strwidth(legend).
text.col the color used for the legend text.
text.font the font used for the legend text
merge logical; if TRUE, merge points and lines but not filled boxes. Defaults to TRUE if there are points and lines.
trace logical; if TRUE, shows how legend does all its magical computations.
plot logical. If FALSE, nothing is plotted but the sizes are returned.
ncol the number of columns in which to set the legend items (default is 1, a vertical legend).
horiz logical; if TRUE, set the legend horizontally rather than vertically (specifying horiz overrides the ncol specification).
title a character string or length-one expression giving a title to be placed at the top of the legend. Other objects will be coerced by **as.graphicsAnnot**.
inset inset distance(s) from the margins as a fraction of the plot region when legend is placed by keyword.
xpd if supplied, a value of the graphical parameter xpd to be used while the legend is being drawn.
title.col color for title.
title.adj horizontal adjustment for title: see the help for par("adj").
seg.len the length of lines drawn to illustrate lty and/or lwd (in units of character widths).

The arguments **x, y and legend** are interpreted in a non-standard way to allow the coordinates to be specified via one or two arguments. If **legend** is missing and y is not numeric, it is assumed that the second argument is intended to be legend and that the first argument specifies the coordinates.
The coordinates can be specified in any way which is accepted by **xy.coords**. If this gives the coordinates of one point, it is used as the top-left coordinate of the rectangle containing the legend. If it gives the coordinates of two points, these specify opposite corners of the rectangle (either pair of corners, in any order).
The location may also be specified by setting x to a single keyword from the list **"bottomright", "bottom", "bottomleft", "left", "topleft", "top", "topright", "right" and "center"**. This places the legend on the inside of the plot frame at the given location. Partial argument matching is used.
The optional **inset** argument specifies how far the legend is inset from the plot margins. If a single value is given, it is used for both margins; if two values are given, the first is used for x- distance, the second for y-distance.
Attribute arguments such as **col, pch, lty**, etc, are recycled if necessary: but **merge** is not recycled. Set entries of **lty** to 0 or set entries of **lwd** to **NA** to suppress lines in corresponding legend entries; set **pch** values to **NA** to suppress points.

Points are drawn after lines in order that they can cover the line with their background color **pt.bg**, if applicable.

Graphs by using built-in data sets

R comes with a number of in-built data sets.The mostly used ones are:

- **mtcars:** The data was extracted from the 1974 Motor Trend US magazine, and comprises fuel consumption and 10 aspects of automobile design and performance for 32 automobiles (1973–74 models)
- **iris**: The data set gives the measurements in centimeters of the variables sepal length, sepal width, petal length and petal width, respectively, for 50 flowers from each of 3 species of iris. The species are Iris setosa, versicolor, and virginica.
- **ToothGrowth:** ToothGrowth data set contains the result from an experiment studying the effect of vitamin C on tooth growth in 60 Guinea pigs
- **PlantGrowth**:Results obtained from an experiment to compare yields (as measured by dried weight of plants) obtained under a control and two different treatment conditions
- **USArrests:**This data set contains statistics about violent crime rates by us state.

Graphs by using built-in data sets–Contd.

We shall now illustrate how to load such a dataset and manipulate the data according to our necessity to generate different types of graphs discussed so far.As each such dataset comprises multiple rows of data with multiple columns, we need to group and summarize the data to make them presentable using the available graphical tools.

Fortunately, there is a package in R having the name **dplyr** package that allows us to quickly group and summarize such datasets.

In this case, our first task is to install the package as shown in the statement below:

install.packages('dplyr')

Our next task is to load the library each time, we require grouping and summarizing for a dataset. It is done with the following statement:

library(dplyr)

We next load the dataset using the statement:

data(name of the desired dataset)

We next use the following basic syntax to group and summarize data:

data %>% group_by(col_name) %>% summarize(summary_name = summary_function)

It may be noted that the functions **summarize()** and **summarise()** are equivalent

Here the special operator **' %>%'** is called **pipeline operator**

Let us now take an illustrative example.

Graphs by using built-in data sets–an illustrative example

Let us make use of the **PlantGrowth** dataset for illustration. Now,having loaded the **dplyr** library, we write the following statements:

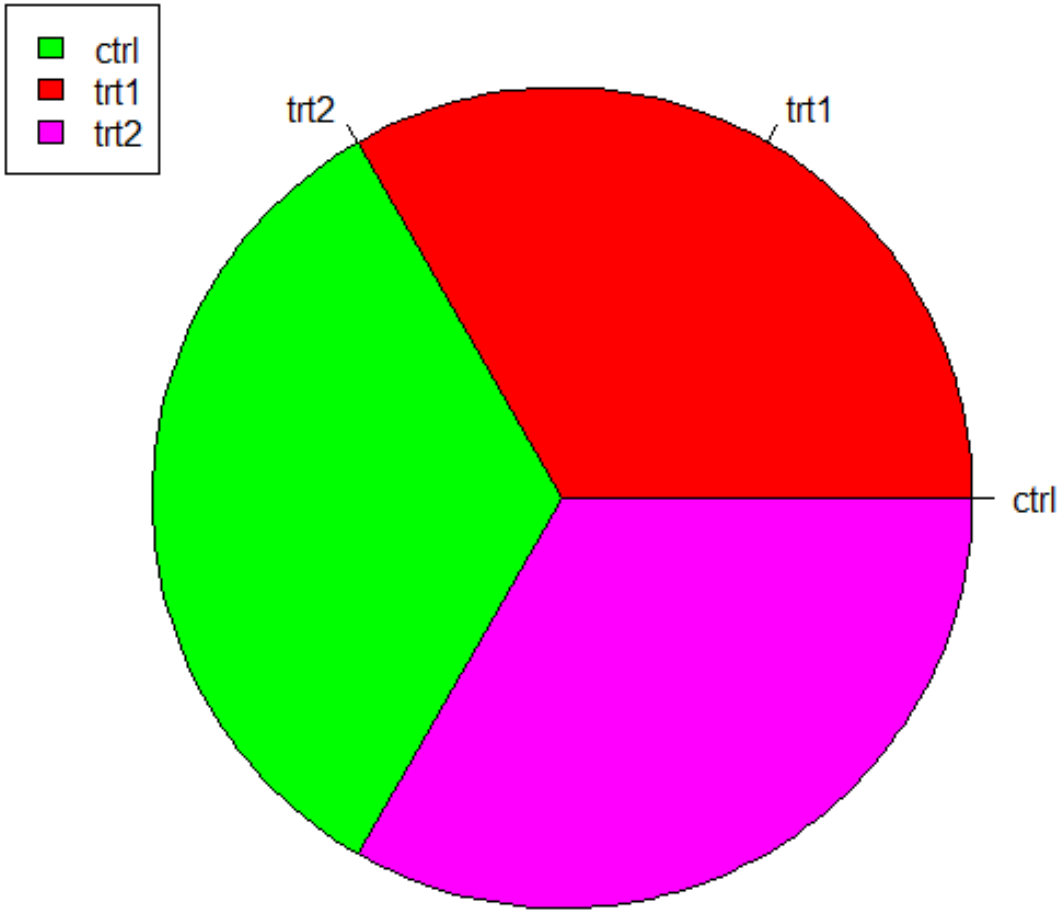
```
>data(PlantGrowth)
>tbl=PlantGrowth %>% group_by(group)%>%
summarize(mean_weight = mean(weight, na.rm = TRUE))
>tbl
# A tibble: 3 x 2
```

	group	mean_weight
	<fct >	<dbl>
1	ctrl	5.03
2	trt1	4.66
3	trt2	5.53

```
>cols=c("green","red","magenta")
>x=table(tbl)
>lbls=table(tbl$group)
>pie(x,labels=lbls,radius=0.9,clockwise=FALSE)
>y=tbl$group
>legend("topright",legend=y,fill=cols)
>pie(x,labels=names(x),radius=0.9,clockwise=FALSE,col=cols)
>legend("topleft",legend=tbl$group,fill=cols)
```

Pie chart with grouped values of a table

This above set of commands generates the following pie chart:

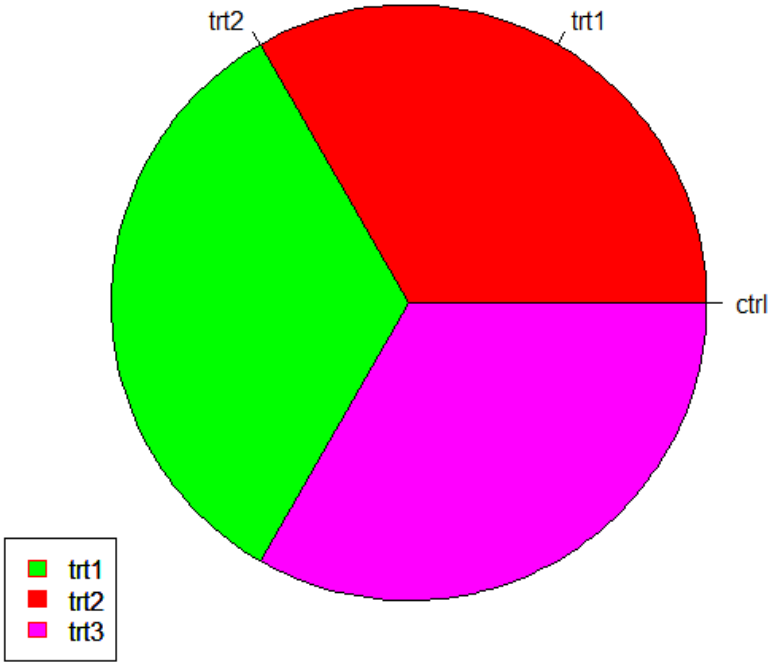


Pie chart with grouped values of a table–Contd.

Let us now consider the following set of commands:

```
>pie(x,labels=names(x),radius=0.9,clockwise=FALSE,col=cols)
>legend("bottomleft",fill=cols,z,border="red")
```

The preceding two commands generate the following pie chart with the col as defined previously.



Grouping table data by Multiple Columns

The group by function can also be used to group data according to more than one feature as well. Let us consider the **mtcars** dataset as stated below:

```
>data(mtcars)
>str(mtcars)
'data.frame': 32 obs. of 11 variables:
$ mpg : num 21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
$ cyl : num 6 6 4 6 8 6 8 4 4 6 ...
$ disp: num 160 160 108 258 360 ...
$ hp : num 110 110 93 110 175 105 245 62 95 123 ...
$ drat: num 3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
$ wt : num 2.62 2.88 2.32 3.21 3.44 ...
$ qsec: num 16.5 17 18.6 19.4 17 ...
$ vs : num 0 0 1 1 0 1 0 1 1 1 ...
$ am : num 1 1 1 0 0 0 0 0 0 0 ...
$ gear: num 4 4 4 3 3 3 3 4 4 4 ...
$ carb: num 4 4 1 1 2 1 4 2 2 4 ...
```

If we consider the **mtcars** data,we can see that the cars have either 1, 2 ,3, 4 ,6 or 8 carburetors. It does the same thing as with one feature, creating groups defined by variables instead of just one variable.

```
>factor (mtcars$carb)
 [1] 4 4 1 1 2 1 4 2 2 4 4 3 3 3 4 4 4 1 2 1 1 2 2 4 2 1 2 2 4 6 8 2
Levels: 1 2 3 4 6 8
```

Further on Group By option with tabular data

Now if we want to know the average mileage of all the cars that use 3 gears and 2 carburetors. The group by function can be used to help us with such information as well. This would require us to add additional columns (i.e., carb) when specifying the input data to the group by function. The implementation would look like the following statement:

>mtcars %>% group_by(gear, carb) %>% summarize(Avg_MPG = mean(mpg))

Functions in summarise()

The verb **summarise()** is compatible with almost all the functions in R. Here is a short list of useful functions which we can use together with **summarise()**:

Objective	Function	Description
Basic	mean()	Average of vector x
	median()	Median of vector x
	sum()	Sum of vector x
variation or Spread	sd()	standard deviation of vector x
	IQR()	Interquartile of vector x
Range	min()	Minimum of vector x
	max()	Maximum of vector x
	quantile()	Quantile of vector x
Position	first()	Used with group_by() to obtain the first observation of the group
	last()	Used with group_by() to obtain the last observation of the group
	nth()	Used with group_by() to obtain the nth observation of the group
Count	n()	Used with group_by() to count the number of rows
	n_distinct()	Used with group_by() to count the number of distinct observations

Histograms

A histogram is a graphical representation of data of a quantitative variable over a continuous interval or certain time period. In a histogram, rectangles are used to display the frequency of the data items in successive numerical intervals of equal size. It is similar in appearance to a bar graph. **It differs from bar graphs in that it groups the values in continuous ranges.Moreover,histograms plot data of one variable at a time only;whereas bar graphs can plot data of multiple variables at a time.** Histograms are useful in showing an estimate as to where values are concentrated or spread, what the extremes are and whether there are any gaps or unusual values. They are also used as a useful tool for getting a rough view of the probability distribution of the dependent variable plotted along the y –axis.

Drawing Histograms

R provides the **hist() function** for creating histograms. The general syntax of the function is as shown below:

hist(x, breaks = "Sturges",freq = NULL, probability = !freq,include.lowest = TRUE, right = TRUE, density = NULL, angle = 45, col = NULL, border = NULL, main = paste("Histogram of" , xname), xlim = range(breaks), ylim = NULL, xlab = xname, ylab, axes = TRUE, plot = TRUE, labels = FALSE, nclass = NULL, warn.unused = TRUE, ...)

Arguments of hist() explained

The arguments in the syntax of **hist()** have the meanings stated below:

x : implies a vector of values for which the histogram is desired.

breaks: implies one of the following:

- a vector giving the breakpoints between histogram cells,
- a function to compute the vector of breakpoints,
- a single number giving the number of cells for the histogram,
- a character string naming an algorithm to compute the number of cells .
- a function to compute the number of cells.

In the last three cases the number is a suggestion only; as the breakpoints will be set toprettyvalues, the number is limited to1e6(with a warning if it was larger). Ifbreaksis a function, thexvector is supplied to it as the only argument (and the number of breaks is only limited by the amount of available memory).

freq : logical; ifTRUE, the histogram graphic is a representation of frequencies, thecountscomponent of the result; ifFALSE, probability densities, componentdensity, are plotted (so that the histogram has a total area of one). Defaults toTRUEif and only ifbreaksare equidistant (and probabilityis not specified).

probability : implies an alias for !freq, for S compatibility.

include.lowest: implies logical; if TRUE, an x[i] equal to the breaksvalue will be included in the first (or last, for right = FALSE) bar. This will be ignored (with a warning) unless breaks is a vector.

right : implies logical; ifTRUE, the histogram cells are right-closed (left open) intervals.

density: implies the density of shading lines, in lines per inch. The default value ofNULLmeans that no shading lines are drawn. Non-positive values ofdensityalso inhibit the drawing of shading lines.

angle : implies the slope of shading lines, given as an angle in degrees (counter-clockwise).

col: implies a colour to be used to fill the bars. The default ofNULLyields unfilled bars.

border: implies the color of the border around the bars. The default is to use the standard foreground color.

main, xlab, ylab : imply the arguments to title.They have useful defaults here.

xlim, ylim : imply the range of x and y values with sensible defaults.It may be noted that **xlim** is not used to define the histogram (breaks), but only for plotting (when plot = TRUE).**axes**: implies logical. If **TRUE (default)**, axes are drawn if the plot is drawn.

plot: implies logical. If **TRUE (default)**, a histogram is plotted, otherwise a list of breaks and counts is returned. In the latter case, a warning is used if (typically graphical) arguments are specified that only apply to theplot = TRUE case.

labels: implies logical or character string. Additionally, it can be used to draw labels on top of bars, if not **FALSE**.

nclass:implies numeric (integer). For S(-PLUS) compatibility only,nclass is equivalent tobreaks for a scalar or character argument.

warn.unused:implies logical **If plot = FALSE** and warn unused = TRUE, a warning will be issued when graphical parameters are passed to **hist.default()**.Finally, **.....** : implies further arguments and graphical parameters passed to plot histogram and thence to title and axis **(if plot = TRUE)**.

Drawing a Histogram for illustration

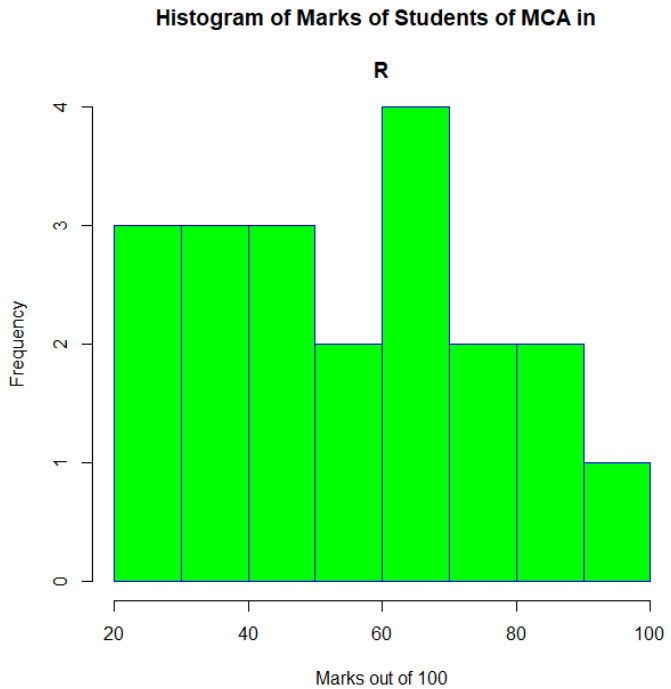
The following data stored in the vector marks_set represent the marks obtained by a batch of 21 students in R programming in a certain institution:

marks_set=c(20,25,30,40,35,38,48,70,55, 60,67,78,89.95,65,44,45,67,78,89,96)

We use the following command to construct the histogram of marks corresponding to the data:

hist(marks_set,main="Histogram of Marks of Students of MCA2 in R", col="green",border="blue",xlab="Marks out of 100")

The figure obtained has been shown below:



Another illustrative example of drawing a histogram

The following command includes marks of 25 students to draw another histogram with more specified parameters as shown below:

```
>R_Marks=c(20,30,45,56,78,89,55,66,68,70,80,
90,95,85,65,75,74,54,59,34,32,22,38,60,70)
>hist(R_Marks,main="Histogram of Marks of Students of BCA in
R",col="green",border="red",
xlim=c(0,100),ylim=c(0,25),breaks=10,xlab="Marks out of 100")
```

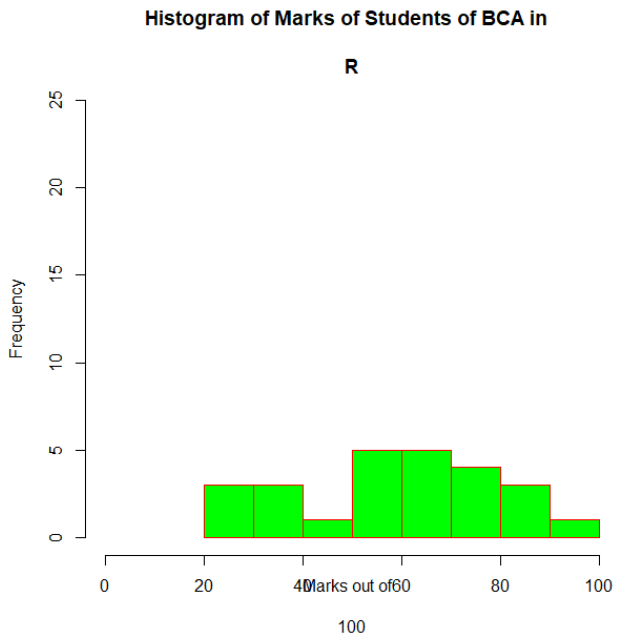


Figure: Another illustrative Histogram

Histograms versus Bar Graphs–A detailed view

Histogram	Bar Graph
A **histogram** is a term that refers to a graphical representation of data using bars to display the frequency of numerical data	A **bar graph** is a graphical representation of data that uses bars to compare different categories of data.
Shows the distribution of non-discrete variables	**Performs** Comparison of discrete variables
Here **the bars touch each other**, so there are no spaces between bars	**Here Bars never touch each other**, so there are spaces between bars
Here the **elements are grouped** so that they are considered as ranges	Here,the elements are taken as **individual entities**
The **width** of Histograms may vary	Bar charts are mostly of equal **width**
This is to **display** the frequency of occurrences	This is to **compare** different categories of data
Here,the **data points are grouped** and rendered based on its range of values, which is known as **class** or **bin**	Here,**each data point** is rendered as a separate bar
The **items** are numbers, which should be categorized to represent data range	Here,**items** should be considered as individual entities
Here,**blocks** cannot be rearranged	Here,it is common to rearrange the blocks, from highest to lowest

The usefulness of histograms:

- Histograms help us to display a large amount of data, which is hard to interpret in a tabular form
- Histograms show the frequency of occurrences of the different data values
- Histograms are useful when calculating the capability of a process
- Histograms help to predict the future performance of the process
- They show the frequency of occurrences of data along with an interval

Histograms versus Bar Graphs–A detailed view–Contd.

The usefulness of bar graphs:

- Bar graphs can be drawn with numerical or categorical data
- bar graphs display each data category in a frequency distribution
- They show the relative numbers or proportions of multiple categories
- They summarize a large data set in visual form.
- They estimate key values at a glance.
- They display close numbers or outline of proportions
- They depict large data set in the visual form and thus help to clarify trends better than tables.

Disadvantages/Drawbacks of Histograms

Here are the cons/drawbacks of histograms:

- They do not allow to read exact values because data is grouped into categories
- They are used only with continuous data
- In Histograms,comparison of data is almost impossible
- Exact measures of central tendencies are not possible because of the use of intervals

Disadvantages of Bar Graphs

Here are the cons/drawbacks of bar graphs:

- Bar graphs display only the frequencies of the elements of data sets.
- Additional explanations are required with Bar graphs to make them understandable
- bar graphs fail to reveal key assumptions, causes, effects, or patterns.

Scatter Plots

A **scatter plot** is a two-dimensional graphical presentation of data that uses dots to represent the values obtained for two different variables - one plotted along the x-axis(called independent variable) and the other plotted along the y-axis(called dependent variable). For example, we may like to plot the number of students opting for higher studies and the percentage of marks obtained at graduation level.Each dot represents one student with his or her percentage obtained at the graduate level measured along the x-axis and option(1 for yes, and 0 for no) measured along the y-axis.

Scatter plots are used when we like to show the **relationship between two variables**. Scatter plots are sometimes called **correlation plots** because they show how two variables are correlated. Not all relationships are linear. For example the plotting of average daily high temperature measured over a month of a year may not be linear as we shall see.We can also have data that show no discernible relationship at all - which itself can be an interesting finding (no correlation).

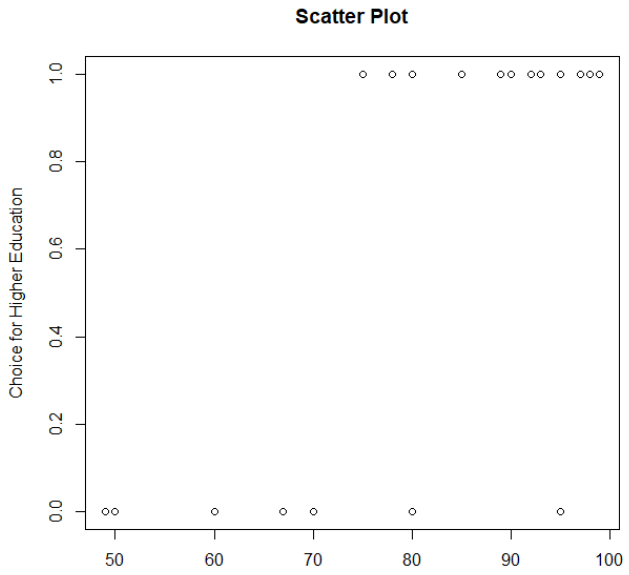
Scatter plots will often include a trendline to help make the relationship more clear.Additionally, the size, shape or color of the dot could represent a third (or even fourth variable). Scatter plots are very useful tools for conveying the relationship between two variables, but we need to know how to use them and interpret them properly.

Drawing Scatter plots

The following data are related to final percentage of 24 graduate students and their choice for higher studies. We use the **plot()** function as stated below to draw the scatter plot shown below:

```
>percentage=c(95,90,92,90,85,70,50,60,49,67,
80,85,89,92,98,99,89,97,95,93,70,80,75,78)
>option=c(1,1,1,1,1,0,0,0,0,0,1,1,1,1,1,1,1,1,0,1,0,0,1,1)
>plot(percentage,option,main="Scatter Plot",
xlab="Percentage",ylab="Choice for Higher Education")
```

The above statement generates the following figure:



Line Charts in R

A **line chart** connects a series of data points with a straight line. Line charts are most often used to visualize data that changes over time. Line charts are usually used in identifying the trends in data.It is used by business firms to monitor closing prices. This is the most basic type of chart used in finance and is created by joining a series of closing prices together. Such charts can be used on any timeframe.
A line chart gives traders a clear visualization of where the price of a security has travelled over a given time period. Because line charts only show closing prices, they reduce noise from less critical times in the trading day, such as the open, high and low. Since, closing prices are typically considered the most important, it is understandable to see why line charts are popular with investors and traders.

Benefits of Using Line Charts:

Clarity: Traders can get confused with too much information when analyzing a security's chart. The trading term **"paralysis by analysis"** describes this phenomenon well.Use of charts that show a plethora of price information and indicators can give multiple signals that lead to complicate trading decisions. Using a line chart helps traders clearly identify key support and resistance levels,trends and recognizable chart patterns.

Limitations of Using Line Charts:

Line Charts–Concepts Contd.

Limited Price Information: Line charts may not provide enough price information for some traders to monitor their trading strategies. Some strategies require prices derived from the open, high and low. For example, a trader may buy a stock if it closes above the high price of the previous 20 days. Also, traders who use more information than just the close do not have enough information to **backtest their trading strategy** by using a simple line chart.

Drawing Line Charts

R provides the **plot( ) function** to create a line graph. The basic syntax to create a line graph is :

plot(v,type,col,xlab,ylab), where,

- **v** is a vector containing the numeric values.
- **type** takes the value "p" to draw only the points, "l" to draw only the lines and "o" to draw both points and lines.
- **xlab** is the label for x axis
- **ylab** is the label for y axis
- **main** contains the title of the chart
- **col** is used to give colors to both the points and lines.

Line Charts–Concepts Contd.

Multiple Lines in a Line Chart

More than one line can be drawn on the same chart by using the lines() function.After the first line is plotted, the **lines() function** can use an additional vector as input to draw the second line in the chart,

We now take an illustrative example as shown below:

>opening_prices=c(dollar=65,yen=45,lira=55,Euro=95)

>plot(opening_prices,type="o",col="green",xlab="Indian Rupee", ylab="Foreign Currencies-dollar,yen,,lira,euro")

The preceding statement generates the chart shown below:

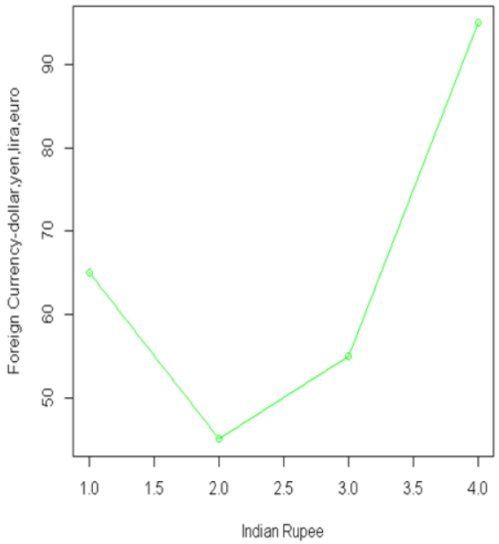


Figure: An illustrative Line Chart

A Line Chart with Multiple Lines

To draw an illustrative line chart with multiple lines, we first draw the preceding line chart and then define another set of values with closing prices as shown below:

closing_prices=c(66,45.5,54.5,52,90)

Now,we use the **lines()** function with a different colour for differentiation as under:

lines(closing_prices,type="o",col="red")

This set of commands generates the following chart with two lines in two different colours:

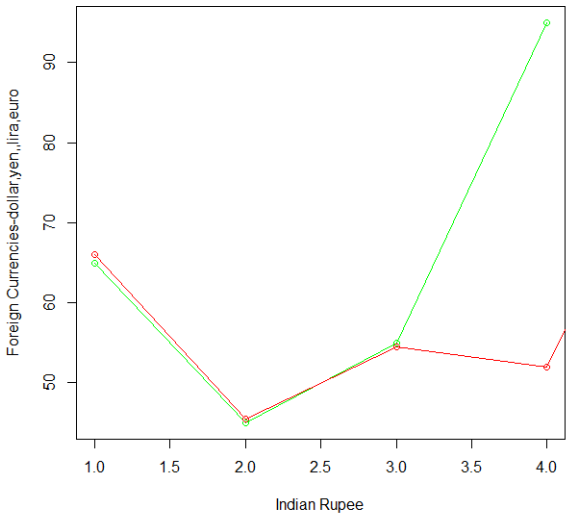
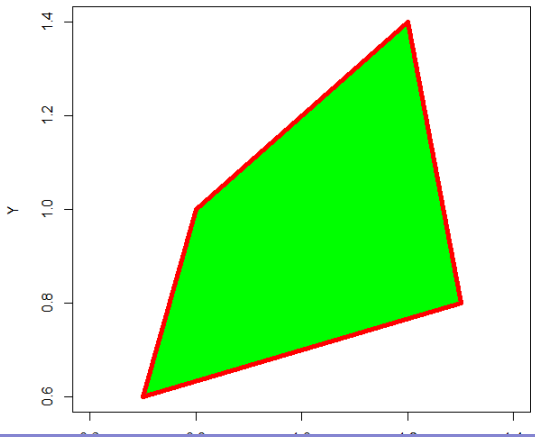


Figure: A Line Chart with two lines

Drawing Polygons

To draw a polygon in R, we need to draw **an empty plot first with a white colour**.We next use the **polygon()** function to draw the desired polygon within the empty plot by using the following basic syntax:
polygon(x_coordinates, y_coordinates,col,border,lwd), where,
'col' takes the color of the polygon
'border' takes the Color of polygon border and
'lwd' represents the thickness of border.
We illustrate the above idea with the following statements:
plot(1, 1, col = "white", xlab = "X", ylab = "Y")
polygon(x = c(0.7, 1.3, 1.2, 0.8), y = c(0.6, 0.8, 1.4, 1), col='green', border='red',lwd=5)
This gives us the polygon shown below:



Drawing Frequency Polygons

We can draw a frequency polygon as stated below:

First, we draw the line of the frequency polygon with the plot function as shown below:

plot(x_coordinates, y_coordinates,type="l",lwd,col)

For illustration, we execute the following statements:

>x1=1:10

>y1=c(2,2.5,3.5,6.5,7.5,8,8.5,9,9.5,10.5)

>plot(x1,y1,type="l",lwd=5,col='red')

This gives us the following line:

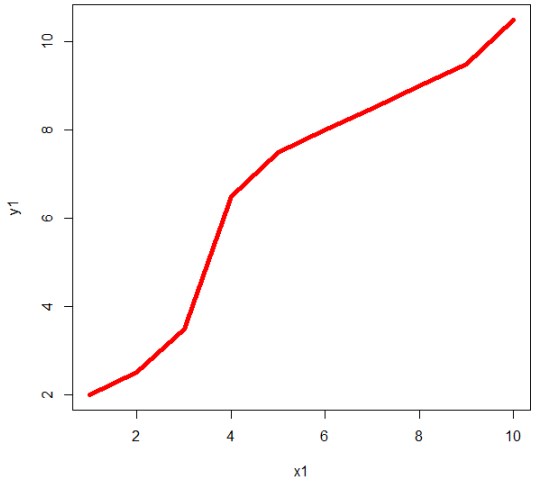


Figure: A frequency Polygon

Ornamenting the frequency polygon

Now, we can add some color below the line with the polygon function:
polygon(x_coordinates, y_coordinates,col,border,lwd) as stated below:
polygon(c(1, x1, 10), c(0, y1, 0),col='green')
This generates the figure below:

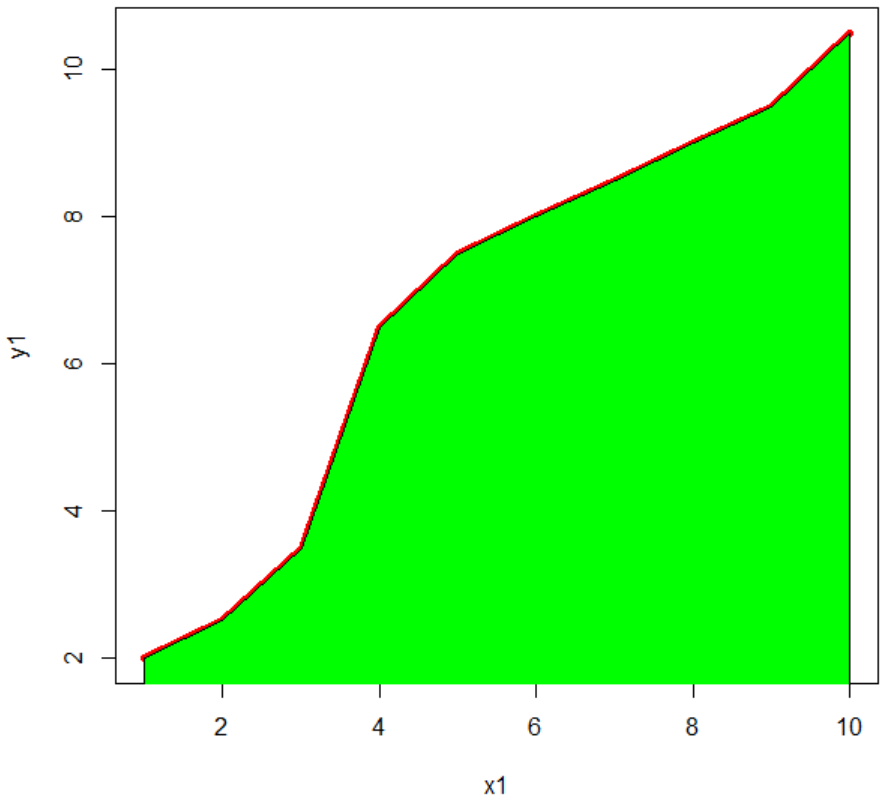


Figure: An ornamented frequency Polygon

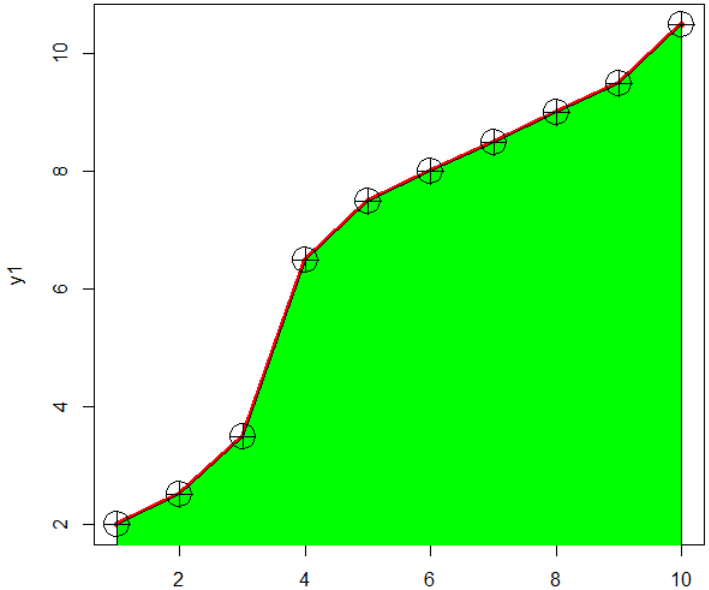
Further ornamentation of the frequency polygon

To make the polygon frequency plot a bit prettier, we can also add some squares/circles at the cut-points of our frequency polygon with a statement like the following: **points(x1, y1, # To add squares to frequency polygon**

cex # To specify the size of point types

pch) # To set the point types:e.g.10 for circles,15 for squares

For example, the statement: **points(x1,y1,cex=3,pch=10)** generates the following figure:



A final touch to the ornamentation

And finally, we can add some line segments to the plot as under:

segments(x1, 0, x1, y1)

This gives us the figure below:

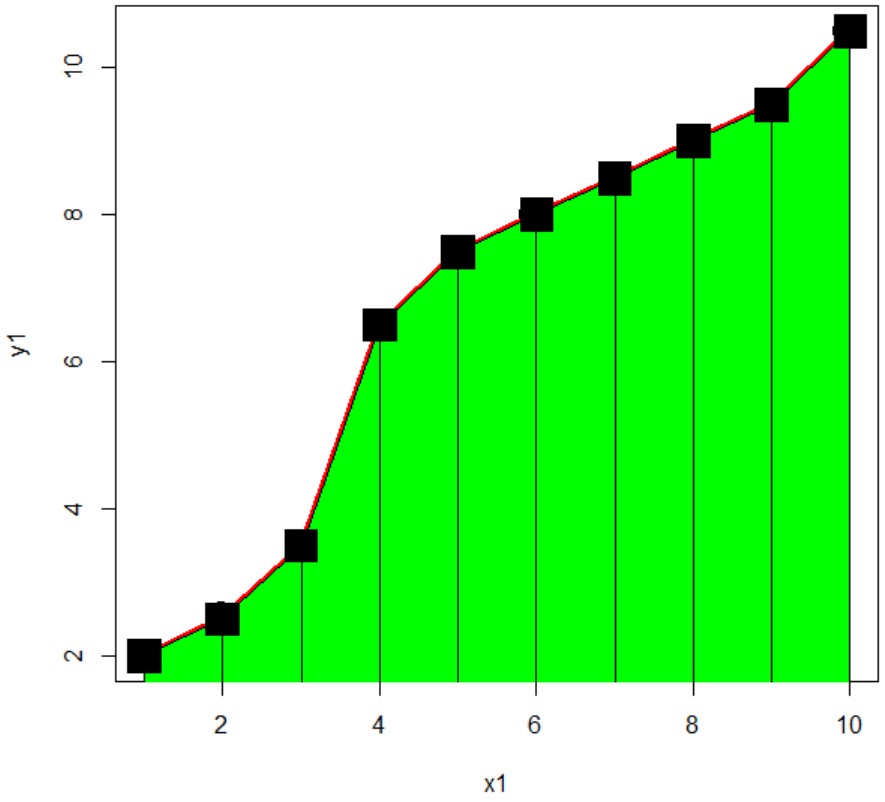


Figure: The frequency Polygon with a finishing touch

Boxplots

A boxplot, also called **a box and whisker plot**, is a type of graphical presentation of quantitative data that shows how well the data in a dataset is distributed by depicting the minimum, maximum, median, first quartile and third quartile in the data set.The median of the data values is pointed out by the vertical line that runs down the center of the box.

The box of a boxplot starts in the first quartile (25%) and ends in the third (75%). Hence, the box represents the 50% of the central data, with a line inside that represents the median. On each side of the box there is drawn a segment to the furthest data without counting boxplot outliers, that in case there exist, will be represented with circles.

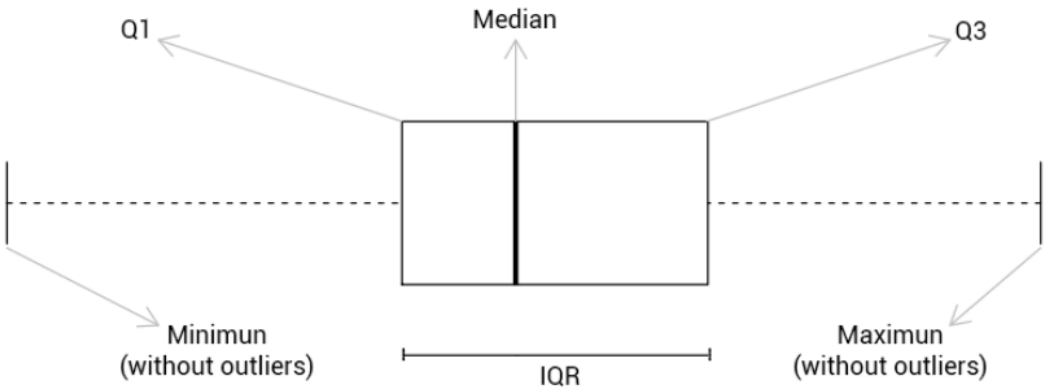


Figure: The structure of a Boxplot

Components of a boxplot

An outlier is an observation that is very distant from the rest of the data. A data point is said to be an outlier if it is greater than $Q_3 + 1.5 * IQR$(Right Outlier) or is less than $Q_1 - 1.5 * IQR$(left Outlier), Q_1 being the first quartile, Q_3 being the third quartile and **IQR** being the interquartile range $(Q_3 - Q_1)$ that represents the width of the box for horizontal boxplots.
So,we observe that **boxplots show two common measures of the variability or spread** in a data set as elaborated below:
Range is represented on a boxplot by the horizontal distance between the smallest value and the largest value, including any outliers and the **Interquartile range (IQR)** is represented by the width of the box (Q_3 minus Q_1).
And finally, **boxplots often provide information about the shape of a data set**.
The examples below show some common patterns.

Some Patterns of Box Plots

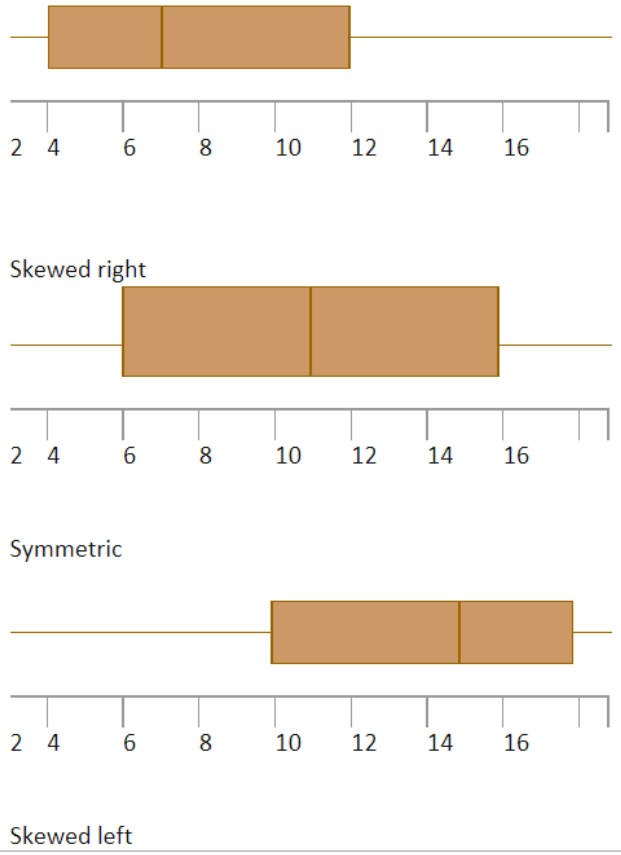


Figure: Boxplots of Different Patterns of skewness

Each of the above boxplots illustrates a different pattern. If most of the observations are concentrated on the low end of the scale, the distribution is skewed right; and vice versa. If a distribution is symmetric, the observations will be evenly split at the median, as shown above in the middle figure.

Drawing Boxplots

In R, boxplot (and whisker plot) is created using the **boxplot()** function. The **boxplot()** function can take any number of numeric vectors, drawing a boxplot for each vector.We can also pass in a list(or a data frame) with numeric vectors as its components.Box plots can be created for individual variables or for variables by group. The basic syntax of the function is:

Boxplot(x, data,notch, varwidth, names, main, col, horizontal)

Where,

- **x** is vector or a formula
- **data** refers to a data frame
- **notch** refers to a v-shaped cut. It is logical and we set it **TRUE** to draw a notched boxplot.
- **varwidth** is logical. We set it to **TRUE** to make the widths of the boxplots proportional to the square root of the sample sizes.
- **names** are the labels of the groups which will be printed under each boxplot.
- **main** is used to give a title to the graph.
- **col** is used to specify the colours of the boxplots
- **horizontal** is logical. It is **FALSE by default** and set to TRUE to reverse the orientation of the axis.

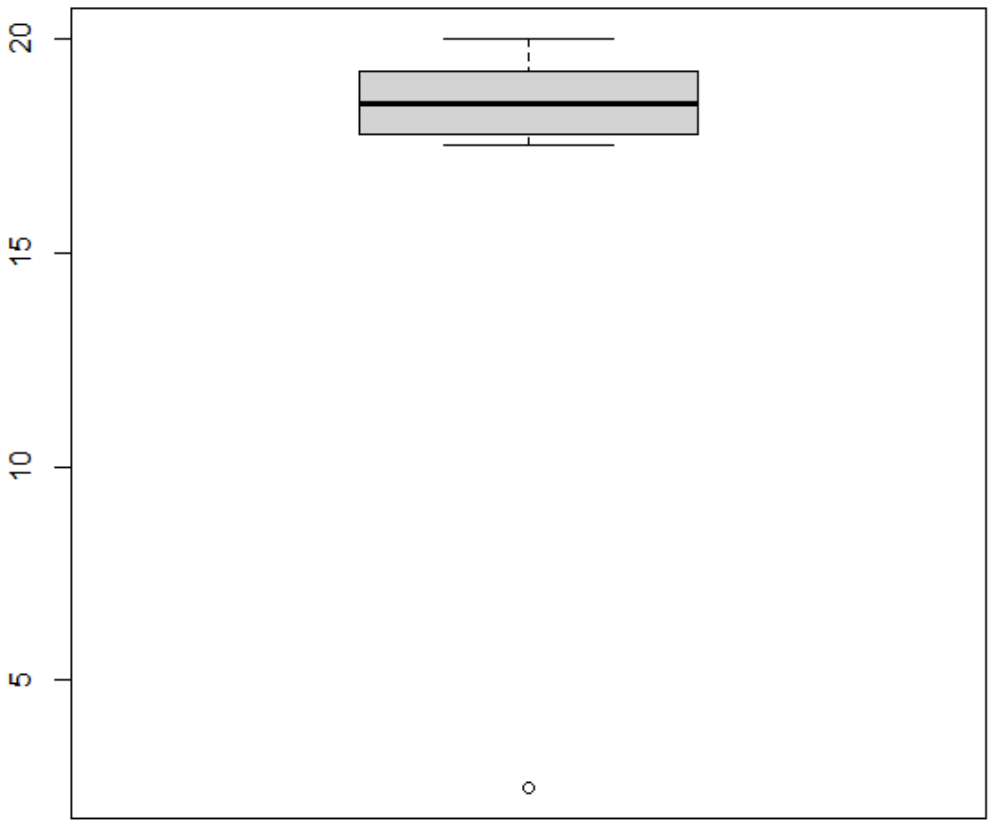
The **Standard Box Plot does not indicate outliers**. The Modified Box Plot highlights outliers. The Modified Box plot is the default in R.

Let us consider drawing a boxplot by taking the age of seven students as shown below:

An illustrative Boxplot from a Vector

```
>age=c(18,19,20,2.5,17.5,18.5,19.5)
>boxplot(age)
```

The following figure shows the generated boxplot:

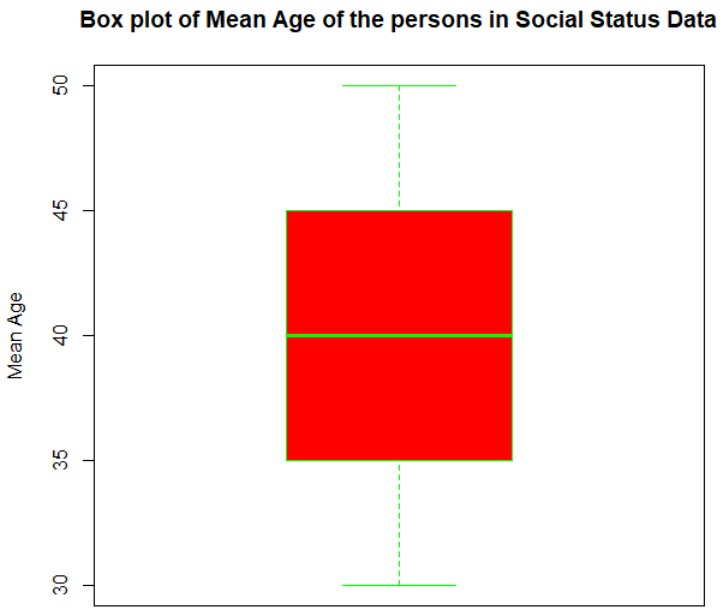


Example Boxplot-2

Let us now consider the data frame that we created earlier while illustrating data frames with the name **social_status**.Let us create a boxplot with the mean age of the persons included in the data frame. The following command can do that for us:

boxplot(social_status$Age,col = "red",ylab="Mean Age", border="green",main="Box plot of Mean Age of the persons in Social Status Data")

The following figure shows the desired boxplot:



Boxplot using some in-built dataset with grouped data

Let us consider the **mtcars** dataset to draw a boxplot. We observe that it comprises the on the attributes shown below:

```
>str(mtcars)
'data.frame': 32 obs. of 11 variables:
 $ mpg : num 21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
 $ cyl : num 6 6 4 6 8 6 8 4 4 6 ...
  disp: num 160 160 108 258 360 ...
 $ hp : num 110 110 93 110 175 105 245 62 95 123 ...
$ drat: num 3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
 $ wt : num 2.62 2.88 2.32 3.21 3.44 ...
 $ qsec: num 16.5 17 18.6 19.4 17 ...
 $ vs : num 0 0 1 1 0 1 0 1 1 1 ...
 $ am : num 1 1 1 0 0 0 0 0 0 0 ...
 $ gear: num 4 4 4 3 3 3 3 4 4 4 ...
 $ carb: num 4 4 1 1 2 1 4 2 2 4 ...
```

Now, we may like to relate the **miles covered by each type of car and the number of Cylinders contained in it** and depict it through a boxplot. We,therefore, use the command below to serve our purpose:

>boxplot(mpg ∼ cyl,data=mtcars,xlab = "Number of Cylinders",ylab = "Miles covered Per Gallon of the fuel",
main = "Data showing the mileage covered",col='green')

Here the data are grouped by number of cylinders contained by a car.

This statement generates the following boxplot:

A boxplot with the mtcars dataset

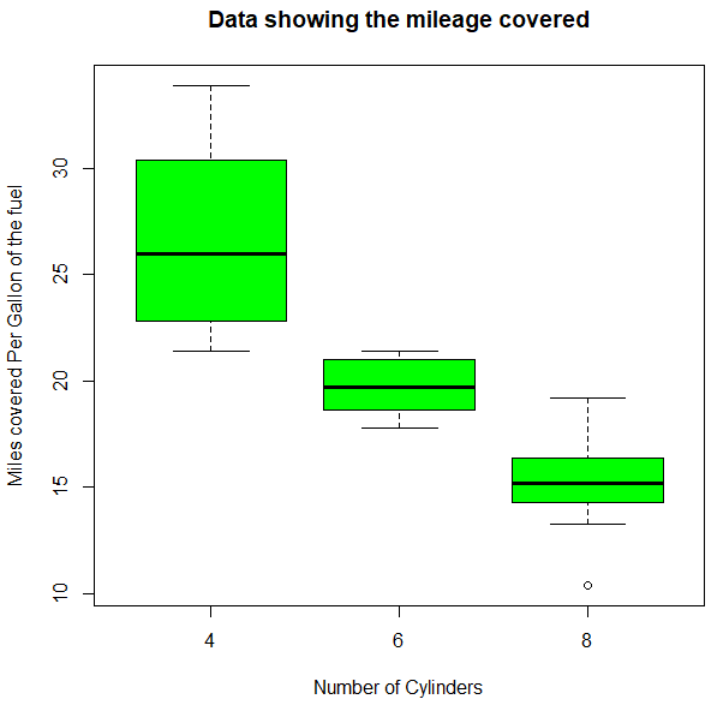


Figure: Boxplot with the mtcars data

We next draw a boxplot using a notch with a view to find out **how the medians of different data groups match with each other**.
We are going to use **xlab as "Number of Cylinders contained by a car"** and **ylab as "Miles covered Per Gallon"**.

A boxplot with notch

>boxplot(mpg ∼ cyl, data = mtcars,xlab = "Number of Cylinders contained by a car",ylab = "Miles Covered Per Gallon", main = "Mileage Data", notch = TRUE, varwidth = TRUE, col = c("yellow", "magenta", "green"),names = c("High", "Medium", "Low"))

This generates the following boxplot

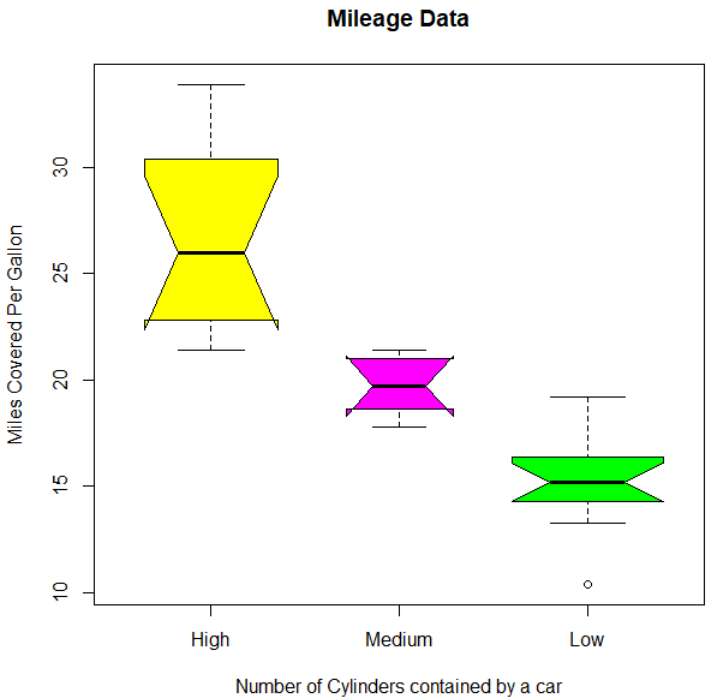


Figure: Boxplot with the mtcars data using notch

Boxplots with the shape of a violin

Boxplots with the shape of a violin are called **violin plots** which are created with the help of **vioplot**() function. This specialized function can be used only if we install the package named **vioplot** and load the corresponding library before using the function as stated below:

>**install.packages("vioplot")**

>**library(vioplot)**

We next Create the data for the **vioplot function** as shown below:

Since,we have cars with 4,6 and 8 cylinders,we shall create a violin for each set of data with the following definitions:

>**x1 = mtcars$mpg[mtcars$cyl == 4]**

>**x2 = mtcars$mpg[mtcars$cyl == 6]**

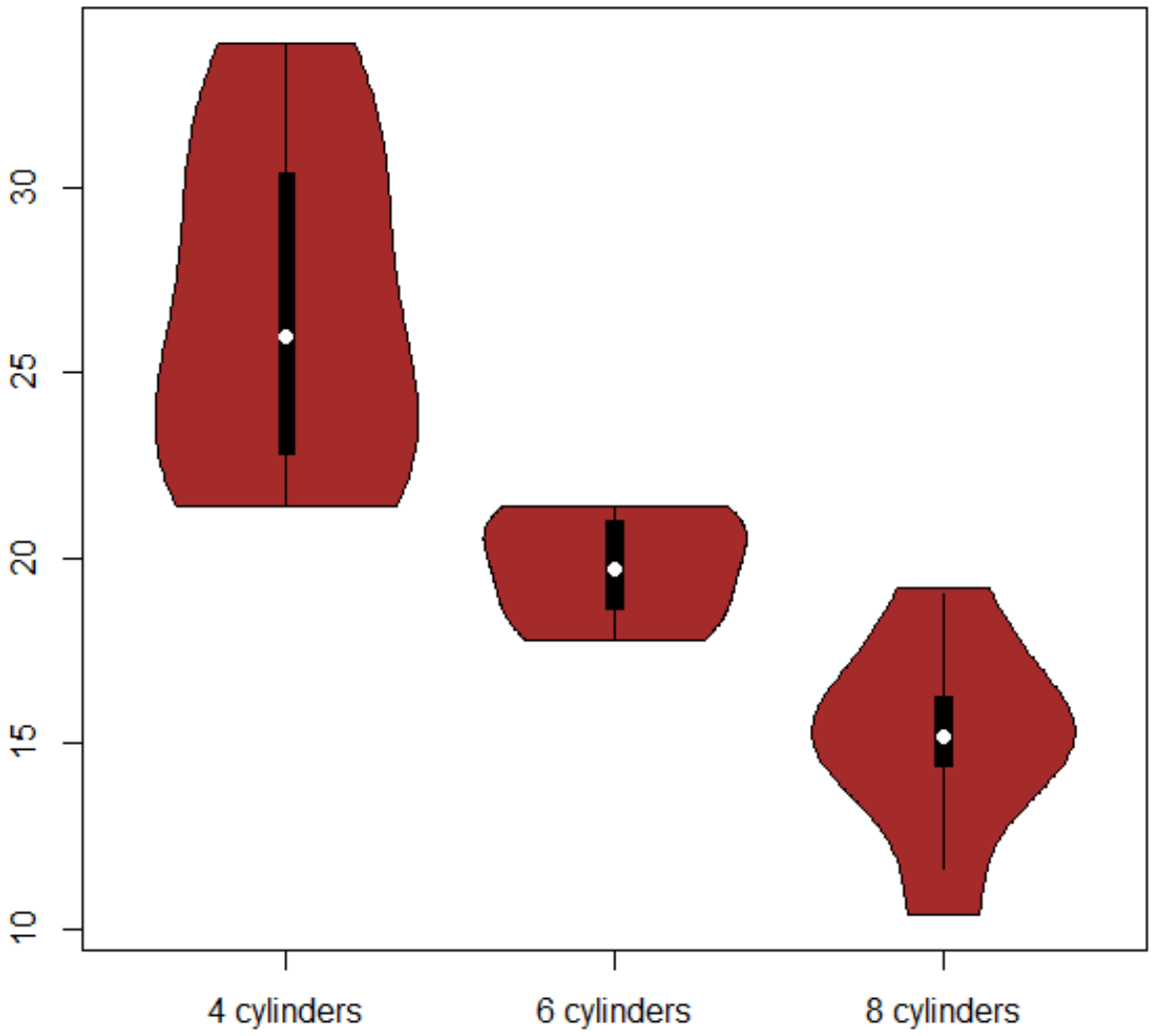
>**x3 = mtcars$mpg[mtcars$cyl == 8]**

Finally, we issue the following command:

>**vioplot(x1, x2, x3,names = c("4 cylinders", "6 cylinders", "8 cylinders"),col ="brown")**

This gives the figure shown below

A Violin Plot



Two-Dimensional Extension of a Boxplot

We install the **aplpack package** for drawing a dimensional boxplot and make use of the**bagplot() function** as shown below:

>**install.packages("aplpack")**

>**attach(mtcars)**

>**bagplot(wt, mpg, xlab ="Car Weight",ylab ="Miles covered Per Gallon",main ="2D Boxplot Extension")**

This generates the following two dimensional extention of a boxplot

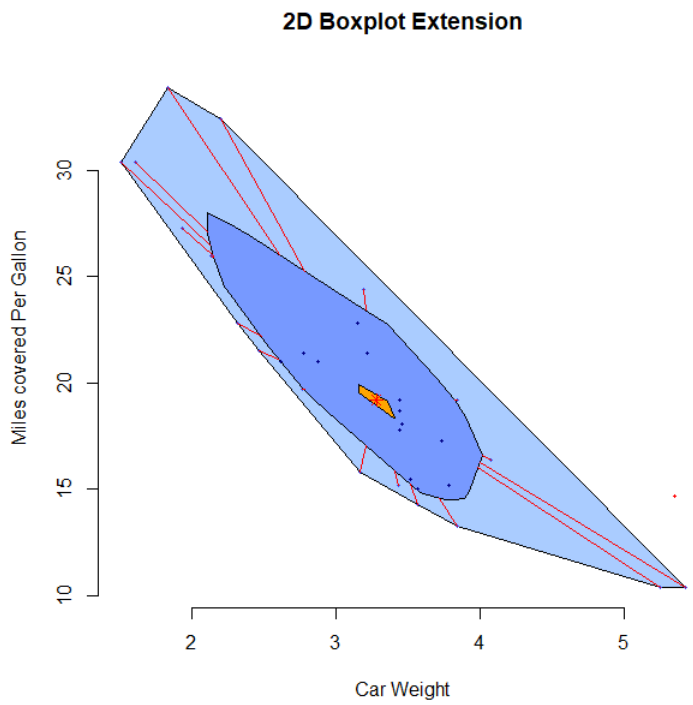


Figure: A Two Dimensional Boxplot

Horizontal Boxplot with points displayed

We have observed that the the boxplot is shown in vertical form,by default.But, we can change the orientation by setting the **horizontal argument to TRUE** as illustrated below with a new vector.

>**v=c(15,18.5,25.6,-23,-45,38.5,39.5,-48)**

>**boxplot(v, horizontal = TRUE,col='green')**

The commands above generates the following boxplot:

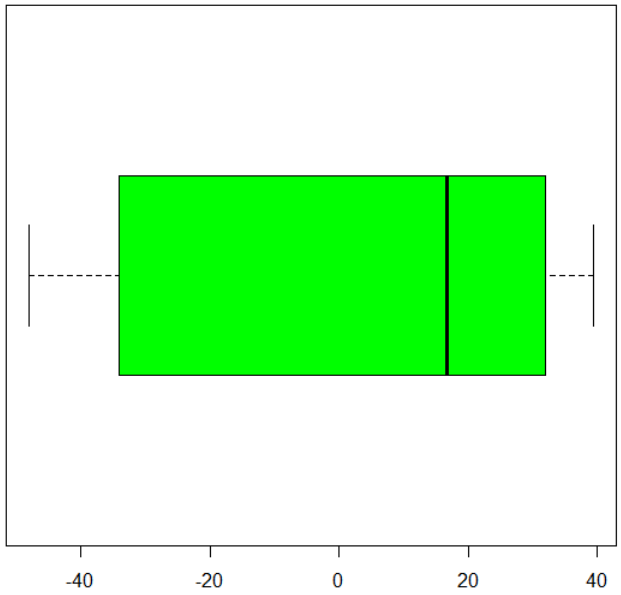


Figure: A boxplot drawn horizontally

Horizontal Boxplot with datapoints

It may be noted that boxplots hide the underlying distribution of the data. In order to solve this issue, we can add points to boxplot in R with the **stripchart() function** (jittered data points will avoid to overplot of the outliers).The following command serves the purpose:

>**stripchart(v, method = "jitter", pch = 19, add = TRUE, col = "red")**

The above statement generates the following figure:

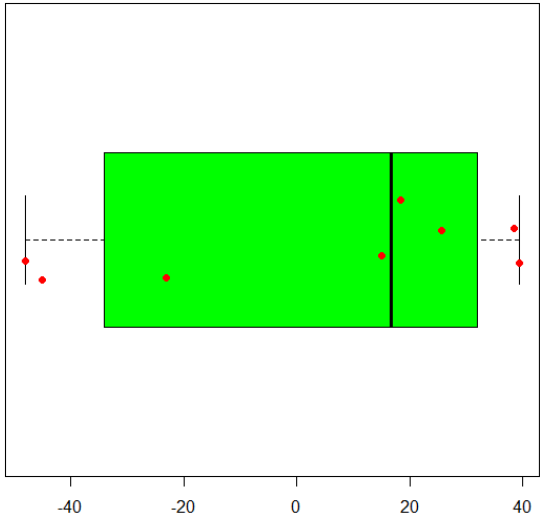


Figure: A boxplot drawn horizontally with data points

Customization of a Boxplot

A boxplot can be fully customized for a nice result.Before this, we would need to know basic syntax of certain functions that would be used here.These are described below briefly.

In order to set graphical parameters, we often use the **par() function** with the following syntax:

par($\cdots$, $no.readonly = FALSE$)

Here, $\cdots$ imply arguments in 'tag = value' form, or a list of tagged values. The tags must come from the names of graphical parameters and no.readonly logical; if TRUE and there are no other arguments, only parameters are returned which can be set by a subsequent par() call on the same device.

For instance, if we show below some uses of par():

>par("usr") gives,

[1] -0.04 1.04 -0.04 1.04

>par("usr")[1:2] gives,

[1] -0.04 1.04

>par("usr")[1] gives,

[1] -0.04

'mfcol, mfrow' implies a vector of the form c(nr, nc).This means that subsequent figures will be drawn in an nr-by-nc array on the device by columns

Functions used in customization contd.

(mfcol), or rows **(mfrow)**, respectively.
The **rect()** R function is used to draw a rectangle (or sequence of rectangles) with the given coordinates, fill and border colors.It has the following syntax:
rect(xleft, ybottom, xright, ytop, density = NULL, angle = 45,col = NA, border = NULL, lty = par("lty"), lwd = par("lwd")
. . .
where,
xleft is a vector (or scalar) of left x positions.
ybottom is a vector (or scalar) of bottom y positions.
xright is a vector (or scalar) of right x positions.
ytop is a vector (or scalar) of top y positions.
density implies the density of shading lines, in lines per inch. The default value of NULL means that no shading lines are drawn. A zero value of density means no shading lines whereas negative values (and NA) suppress shading (and so allow color filling).
angle implies angle (in degrees) of the shading lines.
col implies color(s) to fill or shade the rectangle(s) with. The default NA (or also NULL) means do not fill, i.e., draw transparent rectangles, unless density is specified.
border implies color for rectangle border(s). The default means **par("fg")**. Use border = NA to omit borders. If there are shading lines, border = TRUE means use the same colour for the border as for the shading lines.
lty implies line type for borders and shading; defaults to "solid".
lwd line width for borders and shading. Note that the use of lwd = 0 (as in the examples) is device-dependent.
· · · graphical parameters such as **xpd, lend, ljoin and lmitre** can also be given as arguments.

Next, we show below the syntax of the **invisible() function**.
invisible(x), here x is an arbitrary R object.
This function can be useful when it is desired to have functions return values which can be assigned, but which do not print when they are not assigned.
In the following block of code we demonstrate an example to illustrate how an R box plot can be customized and how to add a grid to it.

```
plot.new()
>set.seed(22)
>rect(par("usr")[1], par("usr")[3], par("usr")[2], par("usr")[4],col = "light grey")
>grid(nx = NULL, ny = NULL, col = "white", lty = 1,lwd = par("lwd"), equilogs = TRUE)
>par(new = TRUE)
# Now, we issue the following command for drawing the Boxplot
```

Customization of a Boxplot–contd.

```
>boxplot(Girth~ Height,lwd = 2, col = rgb(1, 0, 0, alpha = 0.4), xlab =
"Girth of trees",ylab = "Height of the Tree",main = "Customized boxplot with the Trees dataset",notch = TRUE,border =
"black", outpch = 25,outbg = "green", whiskcol = "blue",whisklty = 2, lty = 1)
>legend("topleft",legend = "Boxplot with the Trees Dataset",fill = rgb(1, 0, 0, alpha = 0.4), inset = c(0.03, 0.05),bg =
"white")
```

The generated boxplot has been shown below:

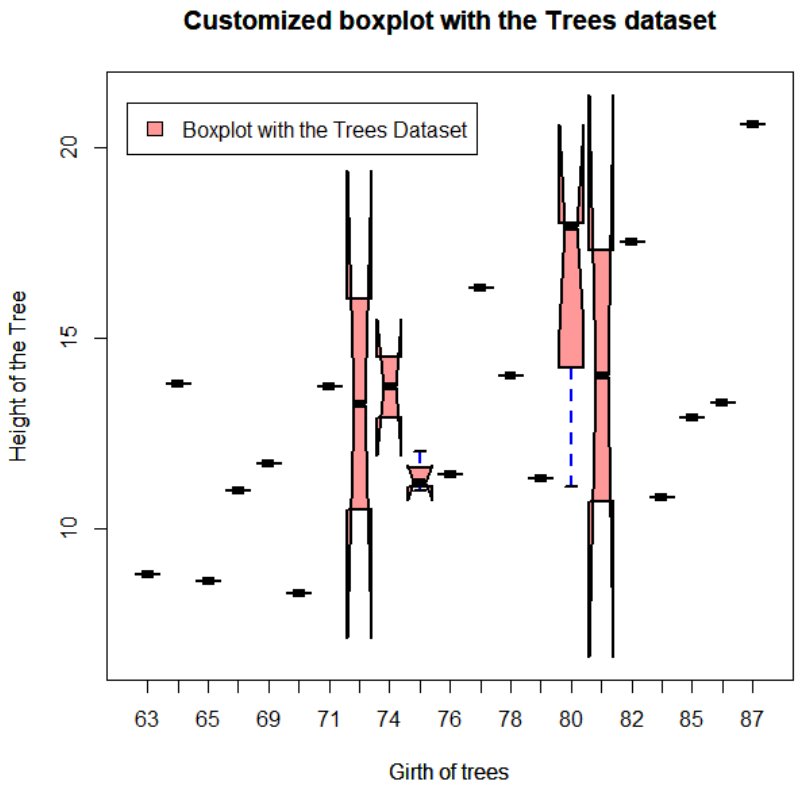


Figure: A boxplot drawn horizontally with data points

Full customization of the boxplot

#Adding mean point to a boxplot in R
By default, when we create a boxplot the median is displayed. Nevertheless, we may also like to display the mean or other characteristic of the data. For that purpose, we need to use the **segments function** if we want to display a line as the median, or the points function to just add points. The code is slightly different for a vertical boxplot from a horizontal boxplot.
In the following code block we show how to add **mean points** and segments to both type of boxplots when working with a single boxplot.
We can combine multiple plots into one overall graph, using either the **par( ) or layout( ) function**. With the **par( ) function**, we can include the **option mfrow=c(nrows, ncols)** to create a matrix of **nrows x ncols** plots that are filled in by **row. mfcol=c(nrows, ncols)** fills in the matrix by columns. So, we use the **par()** below.

```
par(mfrow = c(1, 2))
boxplot(v)
# Adding the mean line
 segments(x0 = 0.8, y0 = mean(v),
 x1 = 1.2, y1 = mean(v),
col = "red", lwd = 2)
 abline(h = mean(v), col = 2, lwd = 2) # Entire line
Adding the mean points
 points(mean(v), col = 3, pch = 19)
 ————————-
# Horizontal boxplot
 ————————-
boxplot(v, horizontal = TRUE)
# Adding the mean line
 segments(x0 = mean(v), y0 = 0.8,
 x1 = mean(v), y1 = 1.2,
col = "red", lwd = 2)
 abline(v = mean(v), col = 2, lwd = 2) # Entire line
# Adding mean point
 points(mean(v), 1, col = 3, pch = 19)
par(mfrow = c(1, 1))
```

Adding mean point and line to a box and whiskers plot
It may be noted that, in this case, the mean and the median are almost equal, as the distribution is symmetric.

Full customization of the boxplot–Contd.

We can change the mean function of the previous code for other function to display other measures.
We can also add the mean point to boxplot by group. In this case, we can make use of the **lapply() function** to avoid **for loops**. In order to calculate the mean for each group we can use the **apply() function** by columns or the **colMeans function**. We can use the code below to add the lines and points for **horizontal and vertical box and whiskers diagrams** together side by side.

```
>set.seed(5) # setting the new seed value for random numbers
>attach(iris) # attaching the in-built dataset iris
>data=iris[,c('Sepal.Length','Petal.Length')]  # Selecting the dataset for drawing boxplots.
>plot.new()
>par(mfrow = c(1, 2)) # Defining the window of the first boxplot to be drawn
>boxplot(data, col = rgb(0, 1, 1, alpha = 0.5)) # Defining the vertical Box Plot
>invisible(lapply(1:ncol(data),function(i) segments(x0 = i - 0.4,y0 = mean(data[, i]),x1 = i + 0.4,y1 = mean(data[, i]),col
= "red", lwd = 2)))
>means = colMeans(data) # This can also be done equivalently as: means=apply(data, 2, mean)
>points(means, col = "red", pch = 19)
>boxplot(data, col = rgb(0, 1, 1, alpha = 0.5),horizontal = TRUE) # Drawing the horizontal Boxplot
>invisible(lapply(1:ncol(data),function(i) segments(x0 = mean(data[, i]), y0 = i - 0.4,x1 = mean(data[, i]),y1 = i + 0.4,col
= "red", lwd = 2)))
>means = colMeans(data)
>points(means, 1:ncol(data), col = "red", pch = 19)
```

The Boxplot as planned above

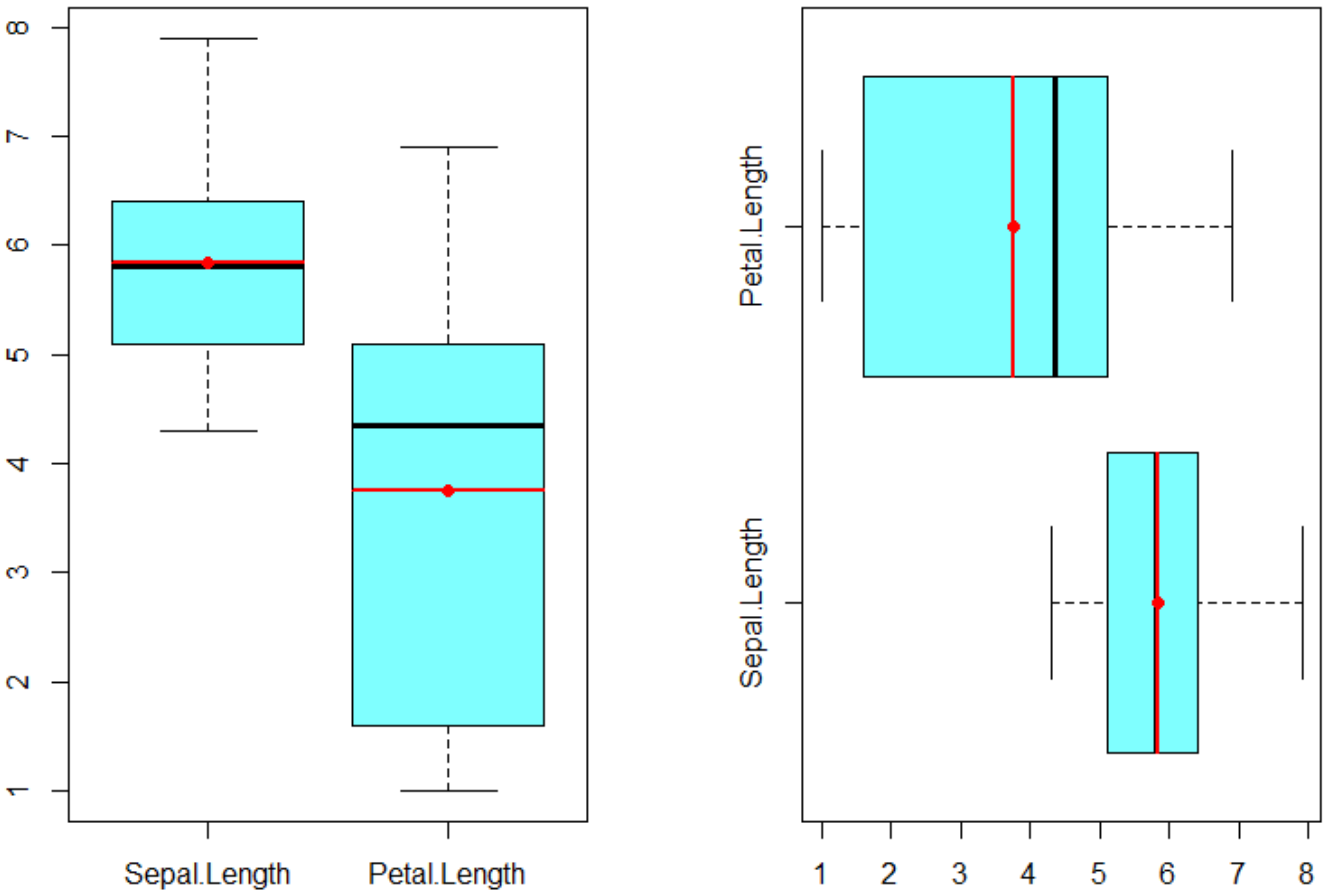


Figure: Two boxplots drawn one horizontally and one vertically with the same set of data points

Return values from boxplot

If we assign the boxplot to a variable,we can return a list with different components.

Let us check this by creating a boxplot with the **iris** dataset and store it in a variable v as shown below:

>**v=boxplot(iris)**

This gives us the following figure:

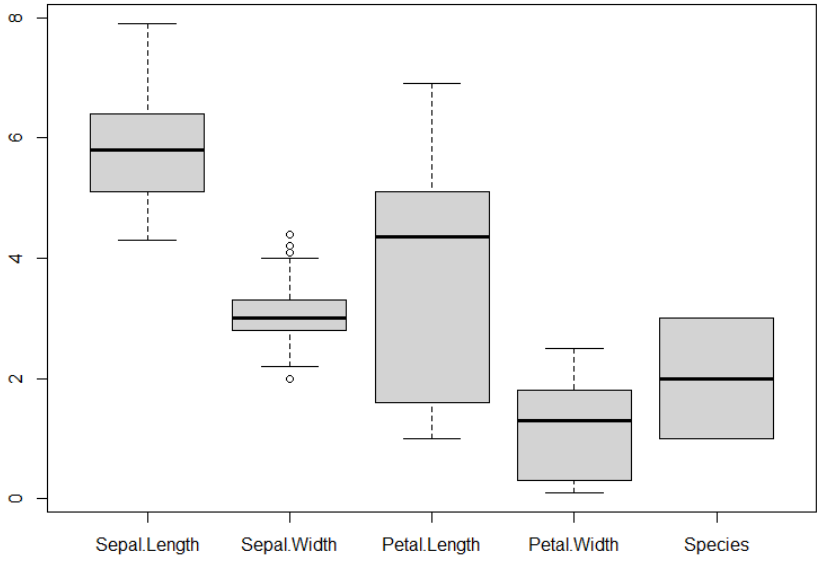


Figure: Boxplot with the whole iris dataset

Return values from boxplot–Contd.

Let us now print the values assigned to v in the preceding statement.

>**v**

This gives the following results:

$stats

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	4.3	2.2	1.00	0.1	1
[2,]	5.1	2.8	1.60	0.3	1
[3,]	5.8	3.0	4.35	1.3	2
[4,]	6.4	3.3	5.10	1.8	3
[5,]	7.9	4.0	6.90	2.5	3

$n

[1] 150 150 150 150 150

$conf

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	5.632292	2.935497	3.898477	1.10649	1.741987
[2,]	5.967708	3.064503	4.801523	1.49351	2.258013

$out

[1] 4.4 4.1 4.2 2.0

$group

[1] 2 2 2 2

$names

[1] "Sepal.Length" "Sepal.Width" "Petal.Length" "Petal.Width" "Species"

Return values from boxplot–Contd.

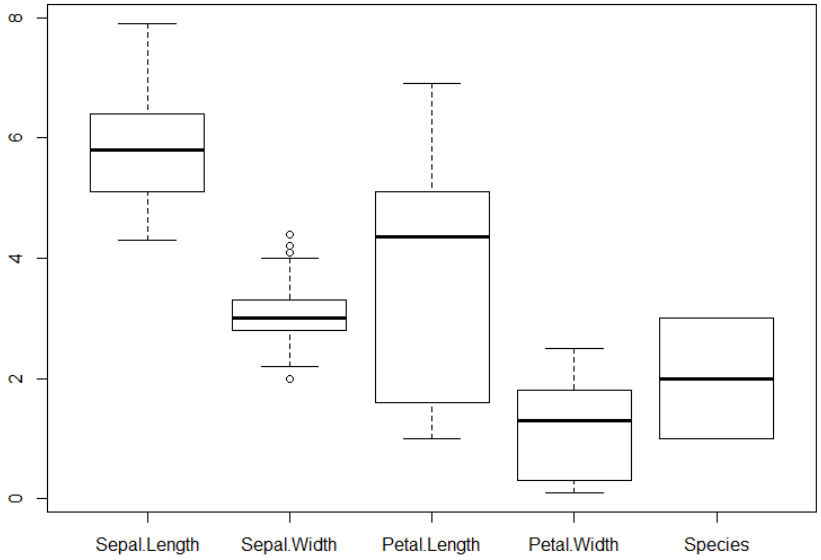
The output will contain six elements described below:

- **stats:** each column represents the lower whisker, the first quartile, the median, the third quartile and the upper whisker of each group.
- **n:** number of observations of each group.
- **conf:** each column represents the lower and upper extremes of the confidence interval of the median.
- **out:** total number of outliers.
- **group:** total number of groups.
- **names:** names of each group.

We can create a boxplot from the variable that has been created (v) with the **bxp function** as shown below:

>bxp(v)

The following figure shows the boxplot generated by this variable.



Boxplot and histogram

One limitation of box plots is that they are not designed to detect **multimodality**. This is why,a boxplot should be combined with a histogram or a density line.The alternative to this problem can be **violin plots or beanplots**.An example is shown below with a set of multimodal dataset.

```
>n=25000
>ber=rbinom(n, 1, 0.5)
>dataset=rnorm(n, mean = 130, sd = 10) * ber +rnorm(n, mean =85,sd=8)*(1-ber)
>hist(dataset, probability = TRUE, ylab = "", col = "green",axes = FALSE, main = "")
>axis(1)
>lines(density(dataset), col = "red", lwd = 3)
>par(new = TRUE)
>boxplot(dataset, horizontal = TRUE, axes = FALSE, lwd = 3, col = rgb(0, 1, 1, alpha = 0.25))
```

The above block of R code will generate the following figure:

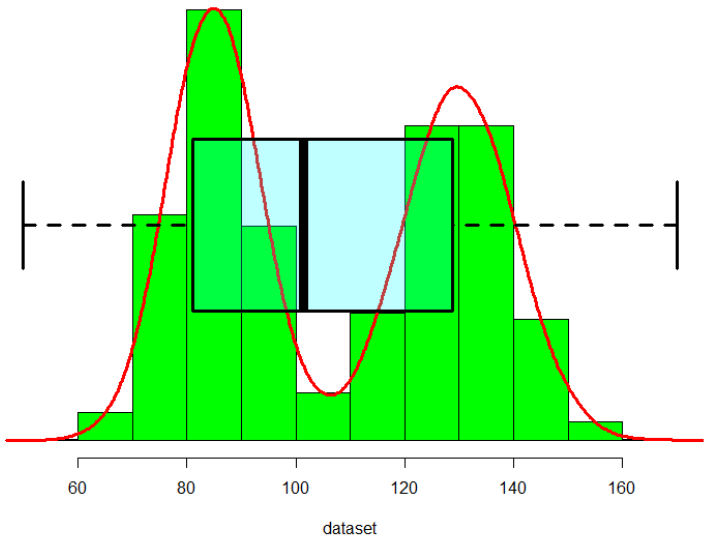


Figure: Boxplot with Histogram and density line

Plotting with ggplot2

ggplot2, also termed as **Grammar of Graphics**, is a free, open-source and easy to use visualization package widely used in R. It is the most powerful visualization package available in R.The plotting package provides helpful commands to create complex plots from data in a data frame. It provides a more programmatic interface for specifying what variables to plot, how they are displayed, and general visual properties. Therefore, we only need minimal changes if the underlying data change or if we decide to change from a bar plot to a scatter plot. This helps in creating publication quality plots with minimal amounts of adjustments and tweaking.

ggplot2 refers to the name of the package itself. When using the package we use the function **ggplot()** to generate the plots, and hence references to using the function is made as **ggplot()** and the package as a whole as **ggplot2**.The plots work best with data in the **'long'** format, i.e., a column for every variable, and a row for every observation.**ggplot2** saves lots of time when making figures with Well-structured data. The graphics of ggplot2 are built layer by layer by adding new elements. Adding layers in this fashion allows for extensive flexibility and customization of plots.

To build a ggplot,the basic format that can be used for different types of plots is as stated below:

ggplot(data = <DATA >, mapping = aes(<MAPPINGS>)) + <GEOM_FUNCTION>()

Plotting with ggplot2–Illustrative Examples

We can start using the ggplot() function and bind the plot to a specific data frame,say **iris**, using the data argument as under:

>**ggplot(data =iris)**

Next, we define an aesthetic mapping (using the aesthetic (aes) function), by selecting the variables to be plotted and specifying how to present them in the graph, e.g., as x/y positions or characteristics such as size, shape, color, etc. as shown below:

>**ggplot(data =iris, mapping = aes(x =Sepal.Length , y =Petal.Length)**

Next, we add **'geoms'** – graphical representations of the data in the plot (points, lines, bars). ggplot2 offers many different **geoms**; the common ones being:

- **geom_point()** for scatter plots, dot plots, etc.
- **geom_boxplot()** for, well, boxplots!
- **geom_line()** for trend lines, time series, etc.

We illustrate **geom_point()** below:

```
>ggplot(data =iris, mapping = aes(x =Sepal.Length , y =Petal.Length))+geom_point()
>ggplot(data =iris, mapping = aes(x =Sepal.Length , y =Petal.Length))+geom_point(alpha=0.5)
>ggplot(data =iris, mapping = aes(x =Sepal.Length , y =Petal.Length))+geom_point(alpha=0.5,color='red')
>ggplot(data =iris, mapping = aes(x =Sepal.Length , y =Petal.Length))+geom_point(alpha=0.5,aes(color=Species))
```

A ggplot with the iris dataset

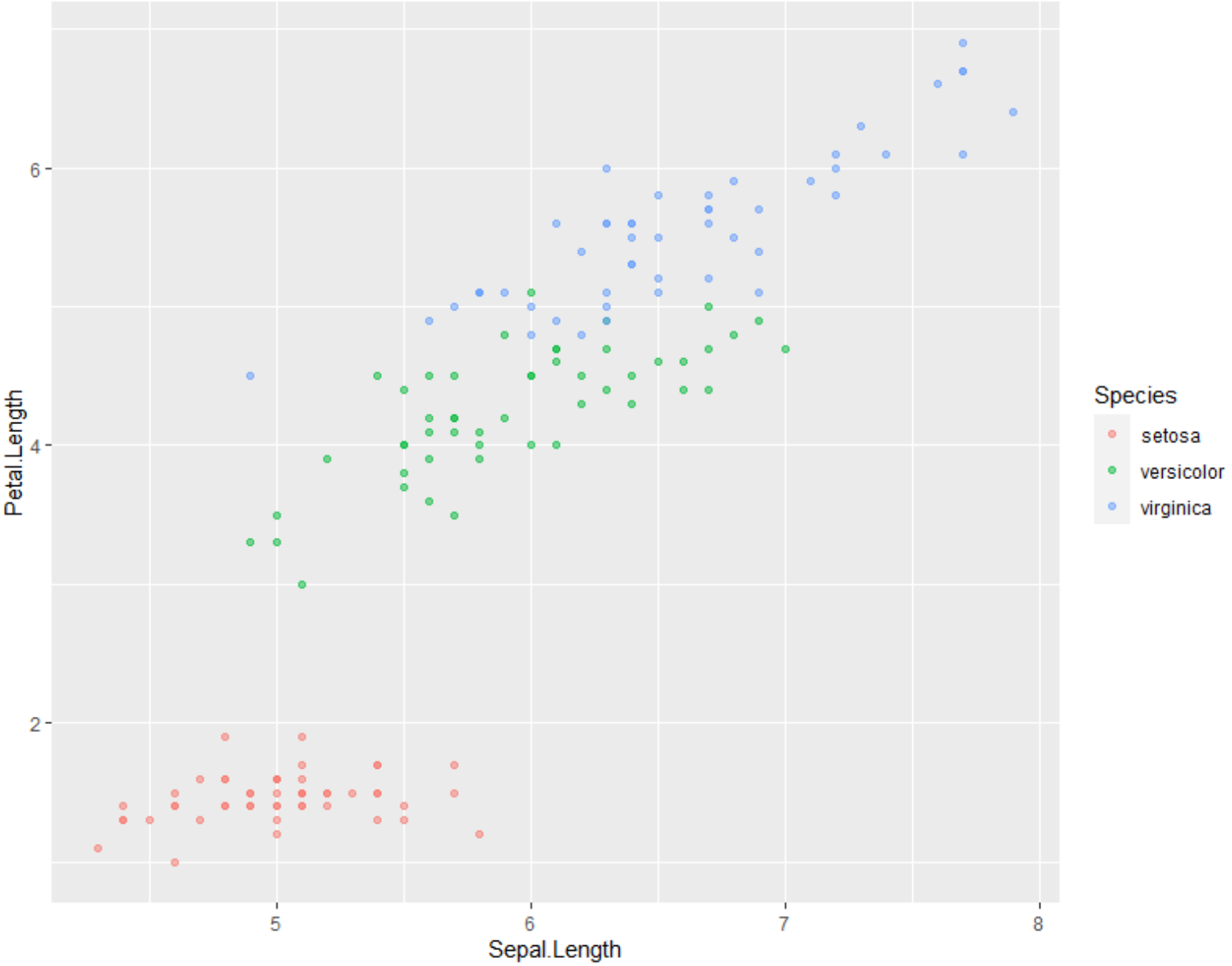


Figure: A ggplot

An illustrative boxplot with ggplot()

We issue the following command to draw a boxplot() with the **iris** dataset:

>ggplot(data =iris, mapping = aes(x =Sepal.Length , y = Petal.Length,group=Species))+geom_boxplot(alpha=0.5, aes(color=Species))

The above statement generates the following boxplot:

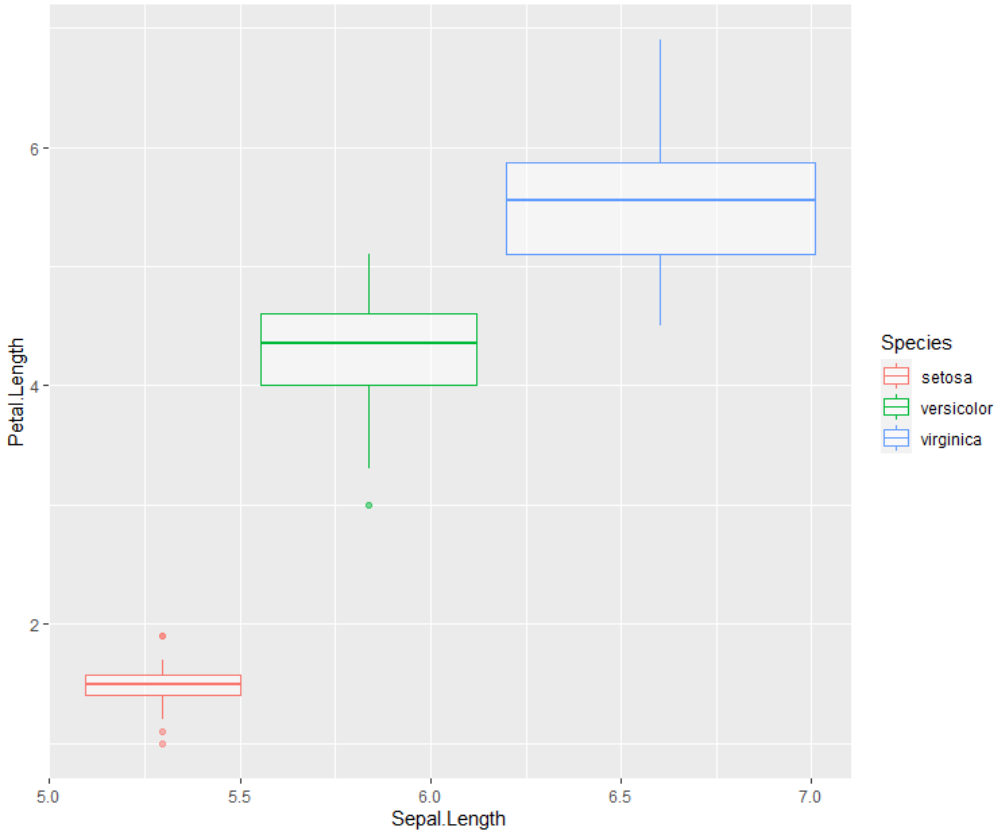


Figure: A boxplot with ggplot()